



Cloud-Based System for detection of Alzheimer's Disease using Deep Learning on MRI Images

Josh Crotty

20096881@mail.wit.ie

Department of Computer Science and Mathematics

B.Sc. (Hons) Software Systems Development

Supervisor: Dr. Bernard Bulter

*Submitted in partial fulfilment of the requirements for the degree of B.Sc. (Hons) Software
Systems Development*

South East Technological University

April 30, 2025

Contents

I. Plagiarism Declaration	5
II. Abstract	6
1 Introduction	7
1.1 Background	7
2 Research Objectives	8
3 Literature Review	9
4 Dataset	12
4.1 Image Preprocessing	13
4.2 Registration Performance and Evaluation	14
5 Methodology	16
5.1 Software System Stack	17
5.2 Workflow	18
5.3 Evaluation and Performance Metrics	19
5.3.1 Evaluation Metrics	19
5.4 Model Architectures	21
5.4.1 ResNet-50	21
5.4.2 DenseNet-201	23
5.4.3 SENet	25
5.5 Performance Comparison of Models	27
6 System Deployment	29
6.1 Application Interface	29
6.2 Model Deployment Architecture	30
7 Tools and Technologies	31
7.1 Part One - Experimentation	31
7.2 Part Two - Deployment	32
8 Project Plan	33
8.1 Semester One	33
8.1.1 Gantt Chart	33
8.2 Semester Two	35
8.2.1 Gantt Chart	35
8.2.2 Homepage Wireframe	36
8.2.3 Patient Page Wireframe	36
8.2.4 Personas and Use Cases	37
9 Web Application Implementation	38

9.1	Homepage	38
9.2	Authentication Page	39
9.3	Patients Page	39
9.4	Patient Inference Page	40
9.5	Model Results	40
9.6	Challenges and Mitigation Strategies	41
10	Conclusion and Future Work	42
11	Bibliography	43
A	Trained Model Figures	45
A.1	ResNet-50 72x72x72 Graphs	46
A.2	ResNet-50 36x36x36 Graphs	48
A.3	ResNet-34 72x72x72 Graphs	50
A.4	DesnseNet-201 72x72x72 Graphs	52
A.5	DesnseNet-201 36x36x36 Graphs	54
A.6	SENet 72x72x72 Graphs	56
A.7	SENet 36x36x36 Graphs	58

List of Figures

1	Unprocessed middle slice from each class label in the axial plane.	12
2	A sample of an MRI slice in three planes (axial, coronal, and sagittal) with preprocessing steps applied. (Row 1) Original MRI; (Row 2) N4 Bias Correction; (Row 3) Affine Transformation; and (Row 4) Skull Stripping.	13
3	Joint Histogram Differences with Reference Image and Registration Methods . . .	15
4	Frameworks and System Architecture that is being used for data analysis and training for this project.	17
5	Diagram of the deep learning system workflow for experimentation and training. .	18
6	ResNet-50 Model Architecture	21
7	Convolutional and Identity Blocks in ResNet-50. These building blocks use skip connections to allow for deeper networks which avoids the degradation problem. .	21
8	ResNet-50 average validation accuracy and ROC curves across 5 folds, showing performance at $72 \times 72 \times 72$ and $36 \times 36 \times 36$ resolutions.	22
9	DenseNet-201 Model Architecture	23
10	Dense Blocks and Transition Layers in DenseNet-201.	23
11	DenseNet-201 average validation accuracy and ROC curves across 5 folds, showing performance at $72 \times 72 \times 72$ and $36 \times 36 \times 36$ resolutions.	24
12	SENet Model Architecture	25
13	Squeeze-and-Excitation Block	25
14	SENet average validation accuracy and ROC curves across 5 folds, showing performance at $72 \times 72 \times 72$ and $36 \times 36 \times 36$ resolutions.	26
15	Deployment Platform diagram using AWS.	29
16	Workload schedule for Semester One, showing weekly task allocations and milestones.	33
17	Workload schedule for Semester Two, showing weekly task allocations and milestones.	35
18	Homepage wireframe for the proposed web application.	36
19	Patient page wireframe for the proposed web application.	36
20	User persona of Dr. Carter who specialises in Alzheimer's Disease and progression.	37
21	User persona of Ms. Sarah Foley who specialises in Medical Imaging.	37
22	User persona of Mr. Joe Blogs with ongoing progression of Alzheimer's Disease. .	37
23	Homepage of web application.	38
24	Authentication page for the application.	39
25	List of patients on the Cognify platform.	39
26	Model Inference on the Cognify platform.	40
27	Model Results from the deployed model.	40
28	Initial EDA Analysis Figures.	45
29	ResNet-50 72 Training Performance Metrics (Part 1)	46
30	ResNet-50 Training Performance Metrics (Part 2)	47
31	ResNet-50 36 resolution Training Performance Metrics (Part 1)	48
32	ResNet-50 36 Training Performance Metrics (Part 2)	49
33	ResNet-34 72 Training Performance Metrics (Part 1)	50
34	ResNet-34 Training Performance Metrics (Part 2)	51

35	DenseNet-201 Training Performance Metrics (Part 1)	52
36	DenseNet-201 Training Performance Metrics (Part 2)	53
37	DenseNet-201 Training Performance Metrics (Part 1)	54
38	DenseNet-201 Training Performance Metrics (Part 2)	55
39	SENet (72) Training Performance Metrics (Part 1)	56
40	SENet (72) Training Performance Metrics (Part 2)	57
41	SENet (36) Training Performance Metrics (Part 1)	58
42	SENet (36) Training Performance Metrics (Part 2)	59

List of Tables

1	Details of the ADNI dataset. CN: cognitively normal; MCI: mild cognitive impairment; AD: Alzheimer's disease.	12
2	Performance metrics of different registration methods. Higher values of Dice, NCC, and SSIM indicate better performance, while lower values of MSE are preferred.	15
3	Test Set Performance Metrics for ResNet-50 & ResNet-34	22
4	Test Set Performance Metrics for DenseNet-201	24
5	Test Set Performance Metrics for SENet	26
6	Model Performance Comparisons with different resolutions	27

I. Plagiarism Declaration

I hereby certify that this assignment is all my own work and contains no plagiarism. By submitting this assignment, I agree to the following terms: Any text, diagrams or other material copied from other sources have been clearly acknowledged and referenced as such in the text by the use of citations including the author's name and date of publication [e.g. (Byrne, 2008)] in the text proceeding the referenced material. These details are then confirmed by a fuller reference in the bibliography. I have read the sections on referencing and plagiarism in the handbook or in the SETU Quality Manual and I understand that only assignments which are free of plagiarism will be awarded marks. I further understand that SETU has a plagiarism policy which can lead to the suspension or permanent expulsion of students in serious cases (SETU, 2023).

Signed: 

Date: April 30, 2025

II. Abstract

This project proposes and addresses a use case for a system for the classification of Alzheimer's Disease (AD) that could be deployed within a cloud environment. Having such a system in place could help with the diagnosis and potentially speed up the procedure of identifying cognitive impairment within patient MRI brain scans. Apart from this, the project also investigates the effects of image resolution and a comparison of deep learning model architectures. The model architectures were compared using evaluation metrics for the task of classifying Alzheimer's Disease (AD), specifically to differentiate between demented (AD) and cognitively normal (CN) patient scans.

A deep learning environment is also developed for the comparison of different model architectures and results will be compared using a cross validation method to give a more robust metric of accuracy. Also, the project gives a walk-through on the deployment for the trained models

1 Introduction

Alzheimer's Disease is a neurological disease that impairs the neural connections and pathways in the brain. It is caused by the abnormal build-up of proteins in and around brain cells. One of the proteins involved is called amyloid, which form deposits of plaque around brain cells, the other protein is called tau, deposits of which form tangles within brain cells [1]. This protein accumulation disrupts cellular function and results in a decrease in neurotransmitter chemicals, which are essential for effective communication between neurons.

Identifying Alzheimer's Disease (AD) symptoms early on is essential for patient screening, diagnosis, and care as it offers patients the opportunity to plan for the future and make appropriate lifestyle changes that could help maintain their quality of life for longer. Early detection of Alzheimer's disease can be difficult in clinical practice due to a number of factors, such as time constraints for clinicians, the difficulty of accurately diagnosing Alzheimer's pathology, and the tendency of patients and healthcare providers to dismiss symptoms as a normal part of ageing [2].

Recently, there has been growing research in applying Machine Learning and Deep Learning techniques to medical imaging. Specifically, advanced imaging modalities such as MRI, PET, and X-rays are being utilised to enhance the classification and identification of diseases. Deep Learning models, in particular, have demonstrated remarkable results, especially through the use of popular architectures like Convolutional Neural Networks (CNNs) [3]. These models leverage their ability to automatically learn hierarchical features from imaging data, significantly improving diagnostic accuracy and efficiency. This project aims to combine both MRI medical imaging of AD and CN patients with Deep Learning models, in order to correctly classify the correct labels for the images.

1.1 Background

Compared to countries like the United States, where healthcare is heavily integrated with technology and supported by robust infrastructure and funding, Ireland's healthcare system has lagged in technological advancements. A study by the Economic and Social Research Institute (ESRI) in 2021 highlighted significant gaps in Ireland's Health Information Systems (HIS) and data infrastructure. Although initiatives like eHealth Ireland [4] have made strides in digital healthcare transformation, they are still limited by funding. In 2021, ESRI estimated that less than 0.8 percent of Ireland's public health budget was allocated to eHealth and health technologies [5]. This figure is considerably lower than in peer nations, where health technology budgets can reach up to 3 percent of total healthcare spending. The funding shortfall inhibits Ireland's capacity to fully leverage health technologies for more efficient patient care and accurate diagnostics, which are critical in areas like Alzheimer's disease diagnosis. Alzheimer's disease (AD) poses a significant challenge to the healthcare sector because early and precise diagnosis is crucial for patient care and planning because symptoms can develop subtly and progress rapidly, timely intervention enables patients to seek early intervention [6]. However, due to the limited investment in healthcare technologies in Ireland, resources for diagnostic tools like machine learning (ML) and deep learning (DL) systems remain underdeveloped. This underinvestment results in a reduced ability to adopt emerging technologies that are transforming AD diagnostics.

2 Research Objectives

The primary objective would be to identify and evaluate suitable model architectures for classifying Alzheimer's disease, along with understanding the effects of image resolution with regard to the models evaluation metrics (accuracy, sensitivity, specificity, etc.).

There are many different model architectures available, such as ResNet, DenseNet, and Vision Transformers (ViT), just to name a few, each offering their own unique learning capabilities. Selecting suitable models, while also comparing the evaluation metrics, can be tricky and requires experimentation. This project will focus on the comparison of selected models along with the effects of augmentation and transfer learning, compiling and analysing each of the metrics to determine the most effective architecture for accurate Alzheimer's disease classification.

To achieve this, the following research questions are proposed:

RQ1

What is the performance impact of specific data transformations (e.g., rotation, scaling, and cropping) on the evaluation metrics (accuracy, sensitivity, specificity, and ROC AUC score) of deep learning models in Alzheimer's disease classification?

RQ2

How do different deep learning architectures (Resnet, DenseNet, variation of a custom hybrid model[s]) compare in terms of accuracy, sensitivity, specificity, and AUC-ROC, when classifying Alzheimer's disease in MRI images?

RQ3

How does varying three-dimensional voxel resolution size affect both model performance metrics (accuracy, sensitivity, specificity, and AUC-ROC) and training efficiency when classifying Alzheimer's disease in MRI images?

The goal of these questions is to show how well different model architectures and classification methods work for Alzheimer's disease, which could lead to improvements in diagnostic approaches.

3 Literature Review

With the rapid advances in deep learning in recent years, Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs) have emerged as focal points in computer vision tasks. This trend was started by Krizhevsky, Sutskever, and Hinton groundbreaking 2012 paper [7], which introduced a deep convolutional neural network capable of classifying 1.3 million high-resolution images in the Large Scale Visual Recognition Challenge 2010 (LSVRC-2010) ImageNet dataset into 1,000 classes. This model significantly outperformed the previous state-of-the-art methods on the same benchmark. This made ImageNet the gold standard for evaluating image classification models. The success of this paper not only sparked the development of other CNN-based models which we will discuss in more detail later, but also marked the beginning of a new era for deep learning research.

A number of new techniques and technologies have been tried and tested in the medical field. Specifically, advances in data analysis and deep learning have enabled the development of accurate, potentially life-saving models that can assist doctors and physicians in diagnosing disease and identifying new health conditions. One such strategy by Reshan et al. [8] implemented a MobileNet CNN-based model to detect pneumonia in X-ray images. The model had 13 million parameters, 28 layers, and the input layer was $224 \times 224 \times 3$. With this model architecture they then trained the model with two datasets, the first consists of 5,856 images which contains 4,273 pneumonia labelled images, and 1,583 normal control labelled images. The second dataset, composed of 1,431 pneumonia labelled images and 1,431 normal control images. After 32 epochs, they achieved a binary classification accuracy of 90.85% and precision and recall were 95.28% and 91.41% respectively.

Another study by Gai et al. [9] compared two prominent deep learning approaches for identifying lung cancer from 3D CT scans: the Data-efficient Image Transformer (DeiT-S) [10] and the EfficientNet3D architectures were used. DeiT-S is a Vision Transformer (ViT) model designed to improve data efficiency by leveraging distillation through attention, enabling it to learn effectively from smaller datasets compared to traditional ViTs. This study is relevant to the current research as it explores the performance of different deep learning models in medical imaging. The findings indicated that while both approaches had advantages under specific conditions, the EfficientNet3D CNN-based architecture outperformed the DeiT-S transformer model. EfficientNet3D achieved the highest accuracy of 93.38% and an AUC of 98.1% when trained with self-supervised learning (SSL), compared to DeiT-S, which achieved an accuracy of 87.75% and an AUC of 95.57%. The authors highlighted limitations of current Vision Transformer (ViT) models in medical imaging, noting that they require significantly larger datasets to perform effectively compared to CNN-based models. This poses a challenge in the medical domain, where large datasets are often unavailable. These findings underscore the suitability of CNN architectures like EfficientNet3D for medical imaging tasks, particularly when data availability is limited.

A study by Jain et al. [11], similar to the work by Gai et al. [9], compared CNN, ResNet, and Vision Transformers (ViT) for multi-classification of chest diseases. The key difference in this study was the use of a much larger dataset. Their dataset consisted of the freely available NIH Chest X-ray dataset, which includes 112,120 X-ray images from 30,805 patients, with disease labels for each image. The authors used a subset of 85,000 images for their performance comparison. The results

showed that both CNN architectures performed similarly, with validation accuracies of 92.68% for CNN and 93.34% for ResNet. Comparing this to the Vision Transformer models, ViT-v1 and ViT-v2, achieved validation accuracies of 92.89% and 92.95%, respectively.

The ViT models used in this study included ViT-v1/32, ViT-v2/32, and ViT-ResNet/16. ViT-v1/32, the base variant, utilizes a 32×32 input patch size, while ViT-v2/32 also uses a 32×32 patch size but with improvements in efficiency and generalisation compared to ViT-v1. The ViT-ResNet/16 variant, on the other hand, uses a smaller 16×16 input patch size and combines the Vision Transformer's attention mechanism with the feature extraction capabilities of ResNet. It was pre-trained on the ImageNet-21k dataset, which is a form of self-supervised learning, offering a large and diverse dataset for pre-training and improving model performance. Despite these differences, ResNet still outperformed the standalone Vision Transformers in terms of validation accuracy.

The authors also introduced a hybrid ViT-ResNet model, which combined the Vision Transformer's state-of-the-art attention mechanism with ResNet's feature extraction capabilities. This model yielded a validation accuracy of 94.07%, similar to ResNet, but with 10% faster training times and a significantly improved ROC curve. The authors highlighted that while CNNs and ResNet provide robust results, ViTs excel in diagnostic accuracy when pre-trained on large datasets such as ImageNet-21k. These findings align with the conclusions in the study by Gai et al. (2024) [9], where the authors also noted that a large dataset is crucial for maximising the performance of ViT models.

The literature that was discussed up to this point above shows the effectiveness and strengths of both CNN-based networks and the newer ViT models. While ViT models generally underperformed compared to CNNs in most cases, the authors attributed this to the limited availability of data for training. Smaller datasets, which are common in medical imaging, pose significant challenges for ViT models that typically require large amounts of data to realise their full potential. CNN-based models demonstrate superior performance on smaller datasets, making them more suitable for medical imaging applications where data scarcity is a common limitation.

The following literature shows the effects of data transformations being applied to training sets and compares evaluation results with and without data transformations and augmentations. Data augmentation is a set of techniques that are used to increase the size and or quality of the dataset by means of image rotations, scaling, random cropping, and variations in pixel values to insert variations within the training dataset. Data augmentation can potentially help in two ways first by generating more data from limited amount of data within smaller datasets such as the medical domain the data used in this project is from and, secondly it helps the model to generalise on unseen data which prevents over fitting.

This [12] study investigates the effectiveness of data augmentation techniques for a standard CNN based sequential model with convolutional, pooling, dropout, flatten, and dense layers. The models input is of 150×150 pixel image dimensions. The architecture is assessed with and without data augmentation methods to determine the impact of data augmentation on model performance. To compare performance, the dataset the authors are using is the Oxford 102 flower dataset which is divided into 102 flower categories, each category containing between 40 and 258 images. In total

the dataset has 8,189 images. The augmentation pipeline the authors are proposing includes four sequential steps which include rescaling, rotating, flipping, and converting to greyscale. The results from the study show improvements to model accuracy by applying the proposed augmentation pipeline to the input images, the models accuracy increased by 3.33%.

The following authors [13] are proposing a new method for the classification of foods. Their paper discusses previously tested methods on food classification but they mention a common problem with other comparative models used in the same domain and that is, food classification can be challenging when dealing with foods that are similar in shape. The authors from this paper propose a high-accuracy food image classification model with data augmentation and feature enhancement through the use of a vision transformer (AlsmViT) which can accurately handle foods with similar shapes. In their paper, they use two food image datasets, Food-101 and Vireo-172, respectively. They present a data augmentation pipeline that includes random resizing and cropping, horizontal flips, and TrivialAugmentWide, which applies a single augmentation to each image. Additional transformations include Gaussian blur to slightly alter image details, greyscale to focus on shape over colour, solarisation to add strong colour noise and focus on shapes, colour jitter to adjust brightness, and finally random erasing, which masks random regions of an image which forces the model to learn features outside the masked area. Their findings concluded that, using the augmentation pipeline, accuracy increased from 89.9% to 95.2% when testing results without the augmentation and transformation on the Food-101 dataset and accuracy improved from 89.2% to 94.3% on the Vireo Food-172 dataset.

One drawback from the data augmentation and transformation techniques discussed above is that they do not include the impact of utilising a pre-trained model alongside data augmentation and transformation techniques. This is essentially true for smaller datasets like that of the study by [12] which uses the Oxford 102 dataset which is relatively small. Exploring the effectiveness of data augmentation and transformation techniques on smaller datasets while also combined with pre-trained models, could yield interesting insights. This raises the question of, how effective are augmentation techniques in improving classification accuracy for medical datasets with limited samples, such as MRI images?

4 Dataset

The dataset used for this project was obtained from the Alzheimer's Disease Neuroimaging Initiative (ADNI) study [14], in which it actively supports the investigation and development of treatments that slow or stop the progression of Alzheimer's disease (AD). Researchers at over 60 clinical sites in the USA and Canada collect data to study the progression of AD in the human brain across normal ageing (CN), mild cognitive impairment (MCI), and Alzheimer's disease and dementia (AD). The ADNI study was first started in 2004 as a public/private partnership led by Principal Investigator Michael W. Weiner. The goal of the ADNI study is to evaluate whether chemicals found in blood, along with magnetic resonance imaging (MRI) and PET scans, can be used together with clinical measures (cognitive and neuropsychological tests), to assess the brain's structure and function over the course of three disease states (cognitively normal/unimpaired, mild cognitive impairment, dementia).

Dataset	Image Scans	CN	MCI	AD	Male	Female	Age
ADNI1: Complete 1Yr 1.5T	2294	705	1113	476	1341	953	75 ± 6.6

Table 1: Details of the ADNI dataset. CN: cognitively normal; MCI: mild cognitive impairment; AD: Alzheimer's disease.

Table 1 shows the unprocessed dataset is quite imbalanced, with 705 CN, 1113 MCI, and 476 AD images for each label, respectively. Because of this, class balancing will be applied to the dataset. The data is converted to use binary (CN, AD) labelling by removing the intermediate MCI data. Initial model development will be a binary classification model which will focus on the CN and AD class labels. Later on, the introduction of the MCI data could be considered for further training and evaluation of model performance.

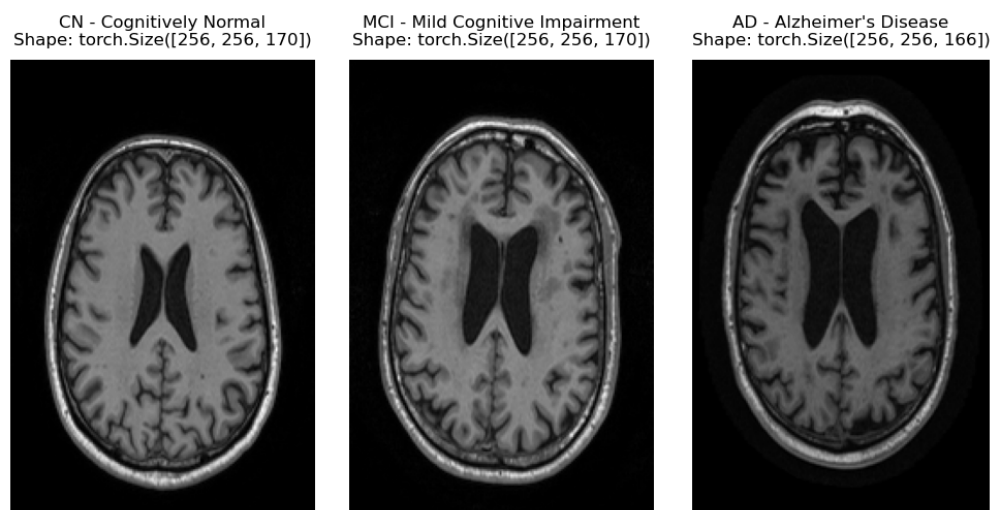


Figure 1: Unprocessed middle slice from each class label in the axial plane.

4.1 Image Preprocessing

There are several preprocessing steps applied to the images before any model can be trained. These preprocessing steps ensure the images are consistently formatted, free of artifacts, and correctly aligned, this will enhance the models ability to learn relevant features. Each of the preprocessing steps will be discussed in the order of application below. Figure 2 shows the output of preprocessing steps.

The bias field is a smooth, low-frequency signal that can distort MRI images by intensity variations in tissues, especially in older MRI machines. It can be caused by various factors such as differences in scanner parameters or variations in patient anatomy. N4 Bias Correction helps to reduce these intensity variations.

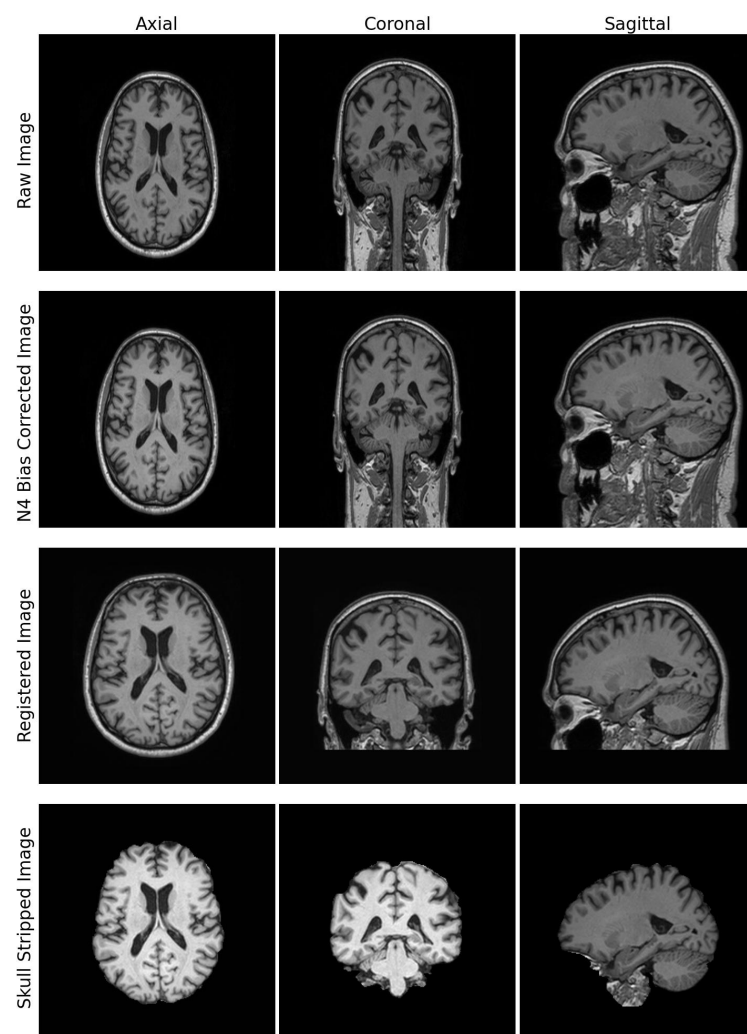


Figure 2: A sample of an MRI slice in three planes (axial, coronal, and sagittal) with preprocessing steps applied. (Row 1) Original MRI; (Row 2) N4 Bias Correction; (Row 3) Affine Transformation; and (Row 4) Skull Stripping.

Image registration is an important component of medical image analysis, it enables structural and functional correlation between images obtained from various modalities, including MRI, CT, and PET. In brain imaging, aligning scans from multiple subjects into a single reference frame is crucial for accurately comparing anatomical data across individuals.

Skull stripping is a technique used to remove non-anatomical structures, such as the skull, from brain images while preserving the brain tissue. This process reduces the amount of data to be analysed and enables deep learning models to concentrate on the relevant anatomical information, enhancing their learning ability.

The FMRIB Software Library (FSL) [15], is a widely used preprocessing tool and offers the capability to apply Affine Registration and Skull Stripping. It was used in this project to ensure precise alignment of the images for effective analysis [16].

4.2 Registration Performance and Evaluation

The goal of registration is to estimate the transformation which maps points from one image to the corresponding points in another image. The transformation estimated via registration is said to map points from the fixed image coordinate system to the moving image coordinate system. In order to apply registration correctly, a reference image is used to map the points. The moving image is the image being registered to the reference coordinate space. For applying registration the reference image being used is the *MNI152.T1_1mm.nii.gz* image from [17] which also aligns with the ADNI1 1mm image voxel sizes.

The performance metrics being used to signify how well an image is transformed into the same coordinate space as the reference image are as follows.

Sørensen Dice Coefficient

The Sørensen Dice Coefficient is a measure of overlap between two sets, commonly used in image registration tasks to evaluate the similarity between a segmented region in the reference image and the segmented region in the registered image.

Formulae 1: Dice Coefficient

$$\text{Dice}(S_1, S_2) = \frac{2|S_1 \cap S_2|}{|S_1| + |S_2|}$$

Where S_1 and S_2 are segmentation masks, $|S_1 \cap S_2|$ represents the intersection, and $|S_1|, |S_2|$ are the sizes of the respective segmentations. A value of 1 indicates perfect overlap, while a value of 0 means there is no overlap between the images.

Normalized Cross Correlation (NCC)

Normalized Cross Correlation (NCC) is a metric used to measure the similarity between two images based on how well they correlate. It normalises the cross-correlation value to make it independent of the image intensity variations, making it useful for rigid and non-rigid image registration tasks.

Structural Similarity Index Measure (SSIM)

Structural Similarity Index (SSIM) is a metric used to assess the perceived quality of images. It evaluates image structures in terms of luminance, contrast, and structural correlation.

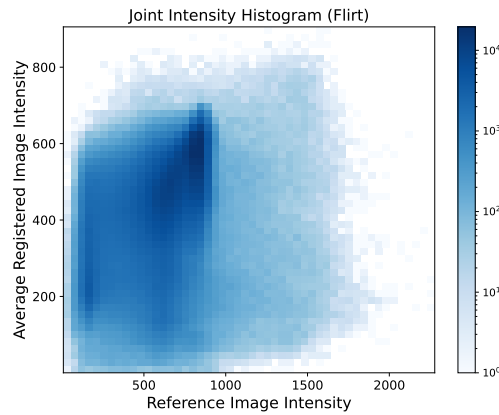
Mean Square Error (MSE)

Mean Squared Error (MSE) measures the average squared differences between corresponding voxel intensities in the reference and transformed images.

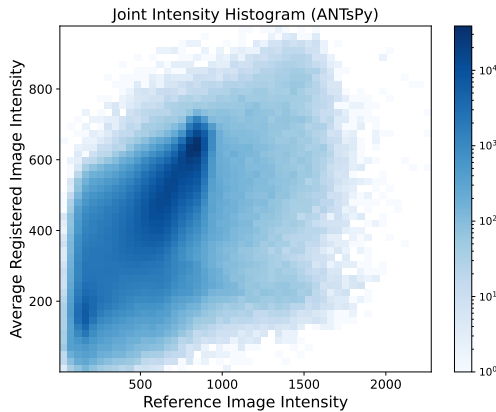
Three frameworks were compared with applying registration to a random sample of 20 images from the ADNI1 dataset [14]. These include, Flirt from the FSL library already discussed in section 4.1, ANTsPy library which is a medical imaging library which also includes registration functionality, and SimpleITK library for segmentation, registration and advanced image analysis.

Registration Method	Dice Coefficient ($\mu \pm \sigma$)	NCC ($\mu \pm \sigma$)	SSIM ($\mu \pm \sigma$)	MSE ($\mu \pm \sigma$)
FSL Flirt	0.864 ± 0.043	0.767 ± 0.086	0.821 ± 0.024	0.006 ± 0.002
ANTsPy	0.918 ± 0.057	0.852 ± 0.095	0.882 ± 0.035	0.004 ± 0.003
SimpleITK	0.745 ± 0.075	0.607 ± 0.101	0.782 ± 0.027	0.010 ± 0.004

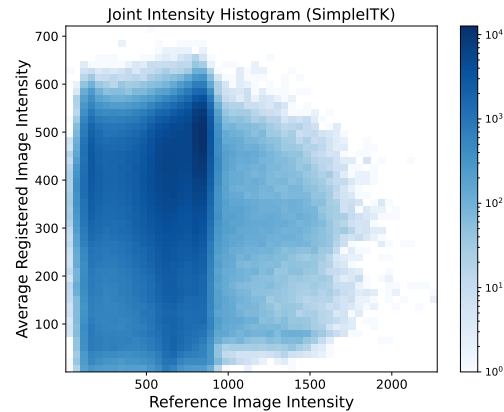
Table 2: Performance metrics of different registration methods. Higher values of Dice, NCC, and SSIM indicate better performance, while lower values of MSE are preferred.



(a) Joint Histogram Difference Between Reference Image and Flirt Registered Images



(b) Joint Histogram Difference Between Reference Image and ANTsPy Registered Images



(c) Joint Histogram Difference Between Reference Image and SimpleITK Registered Images

Figure 3: Joint Histogram Differences with Reference Image and Registration Methods

5 Methodology

This section outlines the methodology used for developing and evaluating the proposed deep learning system for Alzheimer's disease detection. The methodology is structured to provide an overview of the tools, frameworks, and workflows used, along with the design and evaluation of the proposed model architectures.

The section begins with an overview of the software stack used for implementing the deep learning system, including libraries and frameworks. Following this, the workflow of model experimentation and analysis is discussed, accompanied by a diagram to provide a visual representation of the proposed system and its components and how they connect to one another.

Finally, the methodology will finish by presenting the model architectures and evaluation metrics in detail. These metrics, selected based on reviewed research papers from Section 3, align with common practices in medical image classification and general image classification tasks. The selected metrics will form the basis for evaluating the baseline model trained on the proposed dataset.

5.1 Software System Stack

Matplotlib is a low-level graph plotting library in python that serves as a visualisation utility. Matplotlib will be used for plotting model metric results and plotting MRI images. It will also be used for gaining visual insights into the dataset. FSL [15] is a comprehensive library of analysis tools for FMRI, MRI, and diffusion brain imaging data. For this project, the FSL toolkit provides a range of useful tools for preprocessing MRI and FMRI image data.

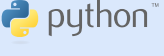
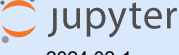
Deep Learning Frameworks	Visualisation & Image Preprocessing
 Version 1.4.0	 6.0.7.14, 8th October 2024
 Cuda version 11.8	 Version 3.9.1
Development Environment	
 Version 3.9	
 2024.02-1	 2024.02-1
Hardware and OS Components	
 Cuda version 11.8	 Build 22H2
CPU - Ryzen 7 5800x 8 Cores	GPU - Nvidia RTX 3080 10Gb
SSD - 1TB Samsung 990 Pro	RAM - 32GB 3600Mhz

Figure 4: Frameworks and System Architecture that is being used for data analysis and training for this project.

control.

The system architecture includes a Windows 10 desktop with 32GB of RAM, a 1TB SSD, and an Nvidia RTX 3080 GPU with 10GB of video memory. The GPU is the main processing unit for training, with CUDA enabling efficient data transfer and batch processing. This setup will accelerate training and reduces processing time for large medical imaging datasets such as the one we will be using (ADNI).

The deep learning frameworks used in this project are PyTorch and MONAI. PyTorch, developed by Facebook's AI Research lab, is an open-source framework that supports CUDA for GPU acceleration, which will improve the training times for large models and datasets. MONAI, built on PyTorch by NVIDIA and MITK, is designed for medical imaging datasets, providing optimised tools for training and inference. By leveraging CUDA, MONAI benefits from faster computations, which is important when it comes to medical imaging such as MRIs as these images can get large.

The development environment consists of Anaconda, Jupyter notebooks, GitHub, and Python. Anaconda simplifies dependency management and integrates necessary libraries, making it easier to work with frameworks like PyTorch and MONAI. Jupyter notebooks offer an interactive environment for coding and experimentation, while Python and GitHub handle programming and version control.

5.2 Workflow

The system workflow follows a structured methodology, guiding the process from the raw dataset through to model evaluations. This approach ensures efficiency and consistency, covering each phase from data preprocessing to result compilation and analysis. The deep learning system, introduced in Section 5.1, provides the foundational software stack for the workflow. The next key component is dataset preprocessing. Before training and analysis, images undergo several preprocessing steps to standardise format, remove artifacts, and align features, optimising them for model training. Details of specific image preprocessing steps was covered in Section 4.1.

Cross-validation is a technique for evaluating model performance on unseen data by dividing the dataset into multiple folds, or subsets. In the testing environment, 5-fold cross-validation will be applied, where one fold serves as the validation set and the model trains on the remaining four folds. This process repeats five times, with each fold taking a turn as the validation set. The evaluation metrics from each iteration are then averaged, providing a more robust estimate of the models overall performance. Any chosen metric can be applied, and averaging across these folds yields a reliable and robust final performance result.

At the end of the workflow, results will be compiled and compared across different data transformations, transfer learning approaches, and model architectures. This comparison will help to address the research questions outlined in Section 2.

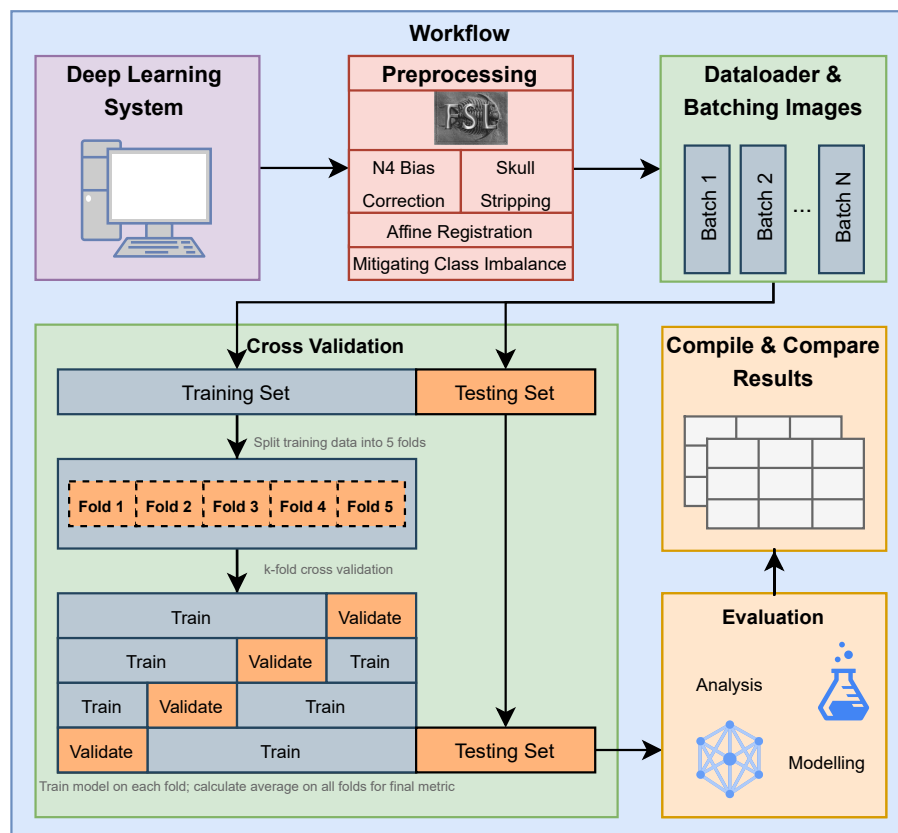


Figure 5: Diagram of the deep learning system workflow for experimentation and training.

5.3 Evaluation and Performance Metrics

Several metrics have been developed to assess the effectiveness of deep learning techniques in medical imaging, tailored to specific tasks such as classification and segmentation. For image segmentation tasks, widely used metrics include the Dice coefficient and Jaccard index, which measure the overlap between predicted and actual segmentations, along with surface distance measures that evaluate boundary accuracy. In image classification tasks, metrics such as accuracy, precision, recall, and F1 score are commonly used to provide a comprehensive view of model performance [18]. These metrics not only gauge overall correctness but also offer insights into each model's ability to minimise false positives and negatives, critical for reliable diagnostics.

5.3.1 Evaluation Metrics

In medical applications, defining the right evaluation metrics is difficult. It is important to minimise both false positives and false negatives, but, a false negative where a condition is incorrectly identified as healthy can have serious consequences such as delayed treatment. Accuracy being a common metric and providing a high-level overview of performance, it may not be the most suitable indicator for medical applications. Accuracy is limited because it measures the percentage of correct predictions, it doesn't check for any errors such as an imbalance of the dataset being used. Accuracy can offer a broad view of model performance, but additional metrics, discussed below, are necessary to better understand the model predictions and ensure it meets the standards in healthcare settings.

Accuracy

Formulae 2: Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Where TP and TN are the numbers of true positives and true negatives, respectively, and FP and FN are the false positives and false negatives.

Recall

Recall, or sensitivity, measures the models ability to correctly identify positive cases, such as patients with Alzheimer's disease (AD). In medical contexts, failing to identify a positive case (false negative) can lead to delayed treatment or insufficient intervention, potentially worsening the patient's condition. Thus, a relatively large recall is crucial for minimising false negatives.

Formulae 3: Recall

$$\text{Recall (or TPR)} = \frac{TP}{TP + FN}$$

Where TP and FN are the numbers of true positives and false negatives, respectively.

Specificity

Specificity, on the other hand, assesses the model's ability to correctly identify negative cases, such as Cognitively Normal (CN) individuals, among all actual CN labels. Together, recall and specificity provide a more nuanced understanding of the models performance.

Formulae 4: Specificity

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Where TN and FP are the numbers of true negatives and false positives, respectively.

AUC-ROC Score

The Area Under the Receiver Operating Characteristic Curve (AUC-ROC) offers a broad measure of the model's discrimination ability. A higher AUC-ROC score indicates that the model is better at distinguishing between Alzheimer's disease (AD) and Cognitively Normal (CN) images, reflecting its overall effectiveness. The ROC curve is a plot of the True Positive Rate (TPR) (also known as recall or sensitivity) against the False Positive Rate (FPR).

The AUC (Area Under the Curve) calculates a single value. An AUC of 1.0 means perfect classification, compared to an AUC of 0.5 where it means no discriminatory ability (no better than random guessing). The formulas for calculating TPR and FPR are shown below.

Formulae 5: Recall

$$\text{Recall (or TPR)} = \frac{TP}{TP + FN}$$

Where TP and FN are the numbers of true positives and false negatives, respectively.

Formulae 6: False Positive Rate

$$\text{FPR} = \frac{FP}{FP + TN}$$

Where FP and TN are the numbers of false positives and true negatives, respectively.

5.4 Model Architectures

5.4.1 ResNet-50

The first model to be trained is ResNet-50 which was developed by He et al. in 2015 [3] at Microsoft Research Asia. Residual networks get their name from the residual blocks that build up a residual network such as ResNet.

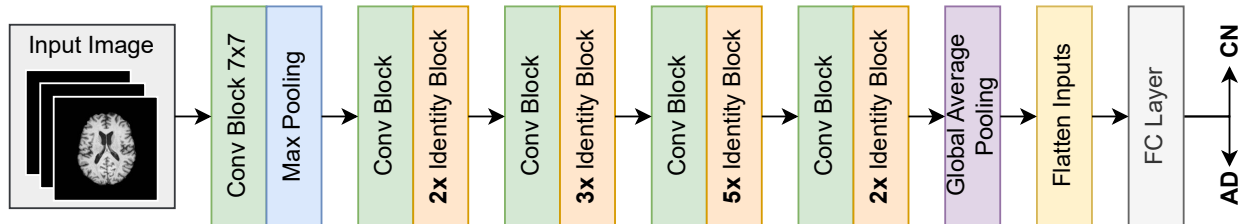


Figure 6: ResNet-50 Model Architecture

ResNet models allow you to train very deep neural networks, avoiding the problem of vanishing gradients by using shortcut connections, which skip one or more layers and help propagate gradients through the network more effectively. Before residual networks, adding more layers to a network did not always improve the model performance, in some cases, it actually led to poorer results. This phenomenon, known as the degradation problem, occurred because deeper networks struggled to learn meaningful representations and gradients would diminish or explode during training, making optimisation difficult.

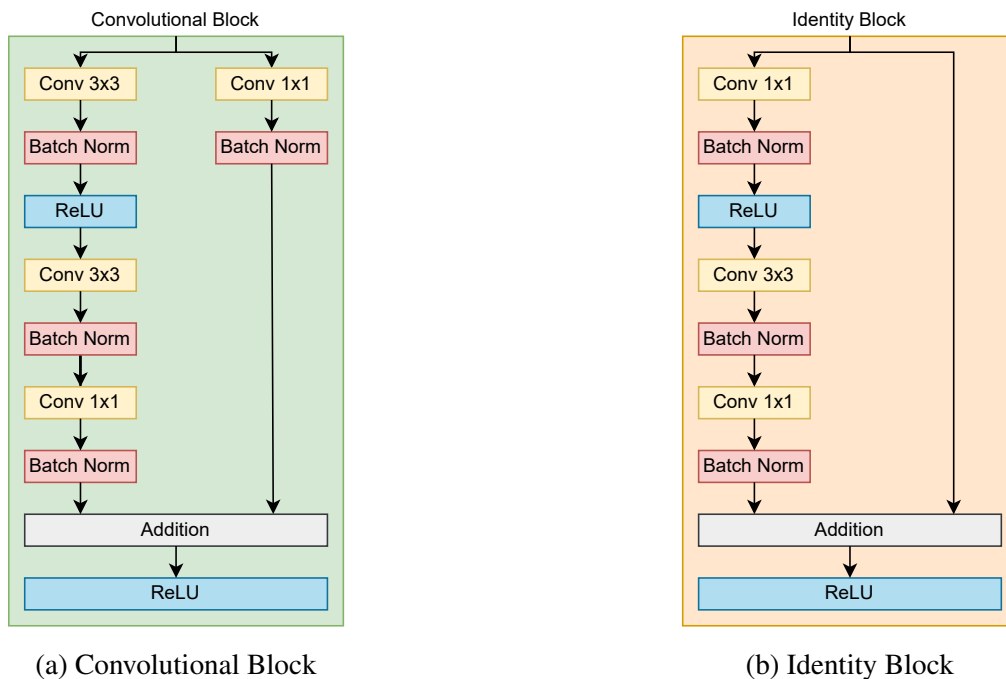


Figure 7: Convolutional and Identity Blocks in ResNet-50. These building blocks use skip connections to allow for deeper networks which avoids the degradation problem.

Resnet Performance Analysis

ResNet-34 was also used to compare the results, as initial ResNet-50 training was over-fitting on the training data. ResNet-34 training time was considerably less while also having better metrics and generalised better to the test set.

Table 3: Test Set Performance Metrics for ResNet-50 & ResNet-34

Resolution	Accuracy	Precision	Recall	AUC Score	Training Time(H)
ResNet-50					
$72 \times 72 \times 72$	76.25	75.63	61.05	79.76	3.50
$36 \times 36 \times 36$	72.64	71.47	55.44	78.44	1.29
ResNet-34					
$72 \times 72 \times 72$	76.11	74.96	64.56	82.29	1.50

Table 3 compares the performance of ResNet-50 and ResNet-34 architectures across different resolutions on the test set. The ResNet-34 model with $72 \times 72 \times 72$ resolution demonstrated superior performance with an AUC score of 82.29 compared to ResNet-50's 79.76 at the same resolution. This improvement suggests ResNet-34 was less prone to overfitting to the training data. Additionally, ResNet-34 achieved comparable accuracy (76.11% vs 76.25%) while requiring significantly less training time (1.50 hours vs 3.50 hours). This efficiency can be attributed to fewer memory bottlenecks during training, allowing for more optimal batch sizes with the smaller architecture.

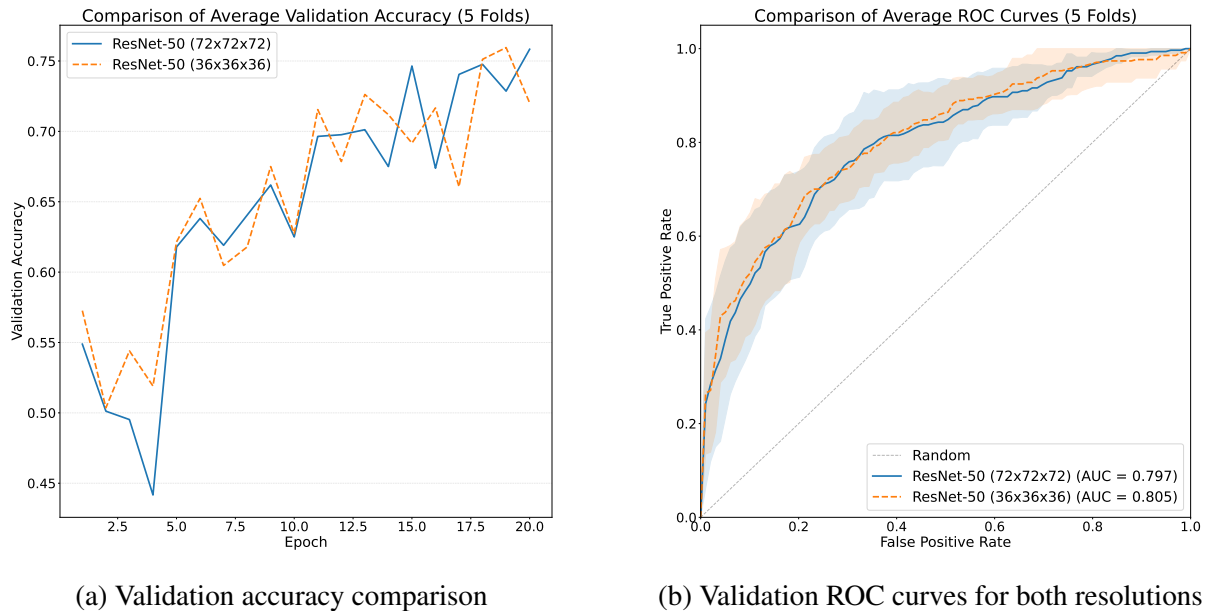


Figure 8: ResNet-50 average validation accuracy and ROC curves across 5 folds, showing performance at $72 \times 72 \times 72$ and $36 \times 36 \times 36$ resolutions.

5.4.2 DenseNet-201

DenseNet-201 was developed by Huang et al. in 2017 and introduces a novel connectivity pattern that connects each layer to every other layer in a feed-forward fashion. In traditional convolutional networks, there are L connections between L layers. In DenseNet, there are $L(L+1)/2$ direct connections, creating a dense connectivity pattern that gives the architecture its name.

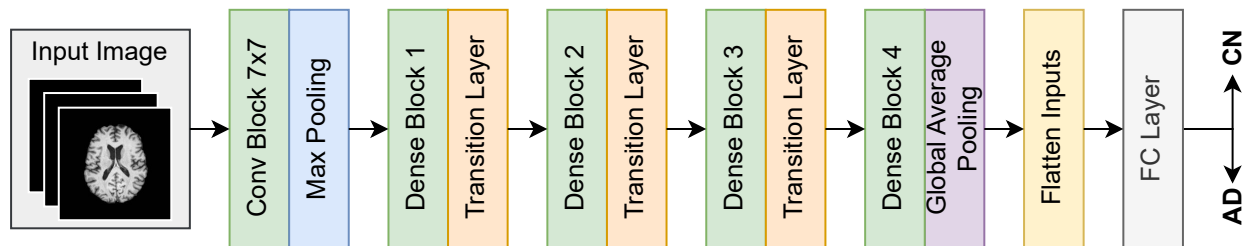


Figure 9: DenseNet-201 Model Architecture

Unlike ResNet which uses shortcut connections to bypass layers, DenseNet directly connects each layer to all subsequent layers. This dense connectivity pattern encourages feature reuse throughout the network and substantially reduces the number of parameters. Each layer receives the feature maps from all preceding layers and passes its own feature maps to all subsequent layers.

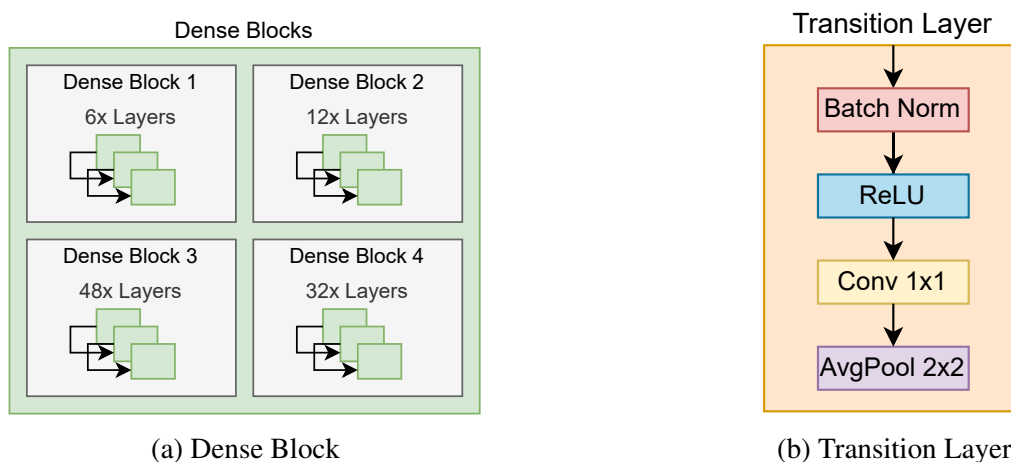


Figure 10: Dense Blocks and Transition Layers in DenseNet-201.

The DenseNet architecture, as illustrated in Figure 9, implements this dense connectivity through specialised components shown in Figure 10. This design allows DenseNet-201 to achieve state-of-the-art performance while using fewer parameters compared to networks of similar depth.

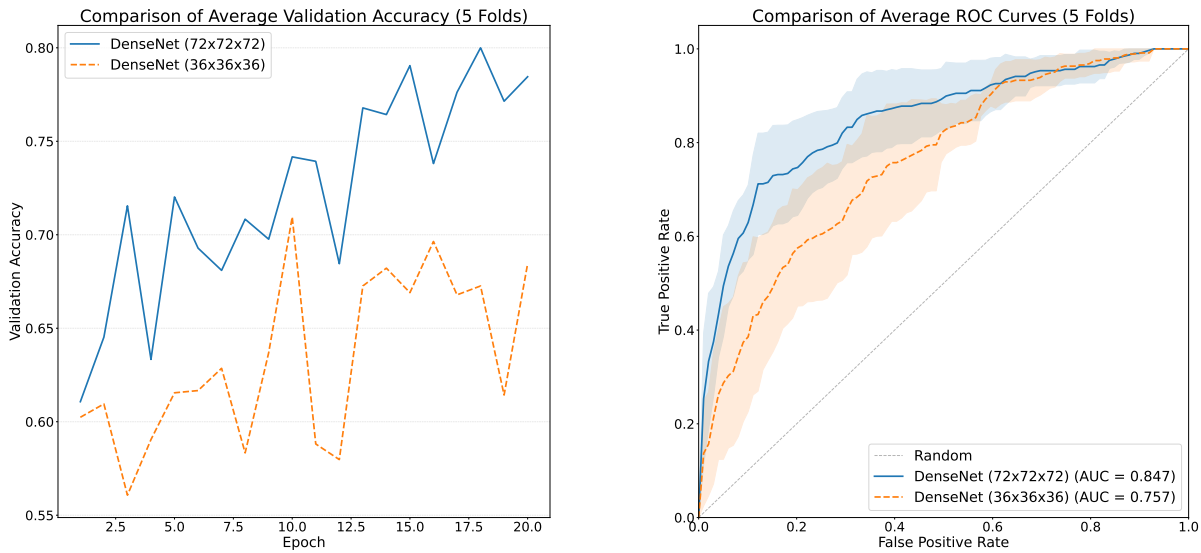
DenseNet-201 Performance Analysis

The performance of DenseNet-201 was evaluated using two different input resolutions: $72 \times 72 \times 72$ and $36 \times 36 \times 36$.

Table 4: Test Set Performance Metrics for DenseNet-201

Resolution	Accuracy	Precision	Recall	AUC Score	Training Time(H)
$72 \times 72 \times 72$	82.22	80.86	72.28	88.61	2.29
$36 \times 36 \times 36$	71.25	69.60	51.58	74.92	2.10

Table 4 provides a comparison of performance metrics across both resolutions on the test set. Key findings include significantly better performance with the $72 \times 72 \times 72$ resolution across all metrics, with an 11% higher accuracy (82.22% vs 71.25%), better precision (80.86% vs 69.60%), and substantially improved recall (72.28% vs 51.58%) compared to the $36 \times 36 \times 36$ resolution. The higher resolution model also achieved a superior AUC score (88.61 vs 74.92), while requiring only marginally longer training time.



(a) Validation accuracy comparison

(b) Validation ROC curves for both resolutions

Figure 11: DenseNet-201 average validation accuracy and ROC curves across 5 folds, showing performance at $72 \times 72 \times 72$ and $36 \times 36 \times 36$ resolutions.

The validation accuracy plot demonstrates Both models are capable of learning patterns but the lower resolution model struggles a little, while the ROC curves indicate good discriminative ability for both image resolutions, but the higher resolution model still outperforms on average. Overall, DenseNet-201 exhibited good performance with the high resolution configuration providing superior results.

5.4.3 SENet

The core idea behind SENet (Squeeze-and-Excitation Network) [19] is to improve channel interdependencies in convolutional neural networks by explicitly modelling the importance of each feature channel. Unlike traditional CNNs that treat all channels equally, SENet introduces a mechanism to adaptively recalibrate channel-wise feature responses.

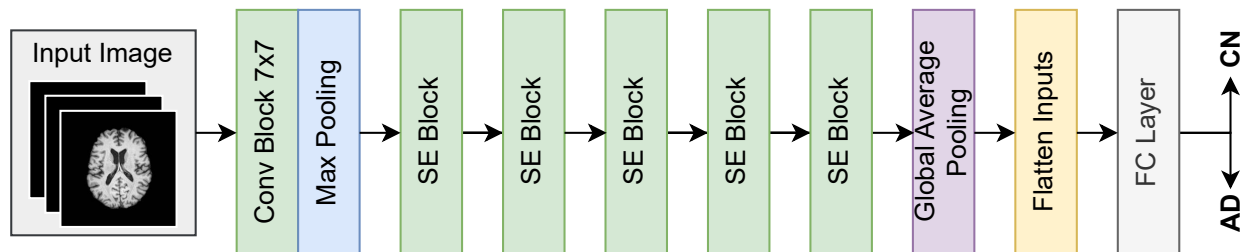
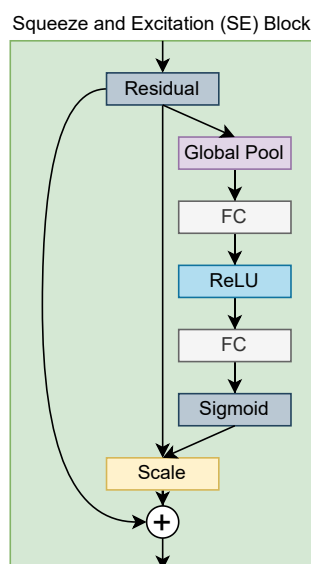


Figure 12: SENet Model Architecture

SENet works by adding a "Squeeze-and-Excitation" (SE) block to existing architectures. The process begins with a squeeze operation that uses global average pooling to encode the entire spatial information into a channel descriptor. This transforms each 2D feature channel into a single value representing global spatial information. Following this, the excitation operation utilises a simple neural network consisting of two fully-connected layers to generate channel-wise attention weights. These weights are then applied to the original feature maps in the scaling step, effectively enhancing important channels while suppressing less useful ones.



ResNet focuses on addressing the gradient vanishing problem using identity shortcuts (skip connections), comparing that to SENet where it enhances ResNet by adding channel attention, which can be inserted into any ResNet block. While ResNet preserves information flow through skip connections, SENet improves the quality of features by emphasising the most informative channels.

The effectiveness of the SENet architecture was conclusively demonstrated when it won the ILSVRC 2017 classification competition. This success highlighted how explicit modelling of channel interdependencies can substantially improve the representational power of convolutional neural networks across diverse computer vision tasks. The SE block essentially allows the network to perform dynamic feature recalibration, adapting the importance of different feature channels based on the input, which enables more nuanced feature extraction tailored to the specific content being processed.

Figure 13:
Squeeze-and-Excitation
Block

SENet Performance Analysis

Table 5: Test Set Performance Metrics for SENet

Resolution	Accuracy	Precision	Recall	AUC Score	Training Time(H)
$72 \times 72 \times 72$	80.28	85.13	62.11	86.76	1.52
$36 \times 36 \times 36$	68.19	67.99	46.32	75.59	1.32

Table 5 provides a comparison of performance metrics across both resolutions on the test set. SENet demonstrated significantly enhanced performance with the $72 \times 72 \times 72$ resolution, achieving 80.28% accuracy compared to 68.19% with the lower resolution which results in a 12% improvement. The higher resolution configuration also delivered better AUC scores (86.76 vs 75.59) while requiring only marginally longer training time (1.52 hours vs 1.32 hours).

SENet’s impressive performance can be attributed to its Squeeze-and-Excitation (SE) blocks, which enhance the representational power of the network by explicitly modelling channel interdependencies. Unlike its close counterpart ResNet, SENet adaptively recalibrates channel-wise feature responses, allowing it to selectively emphasise informative features while suppressing less useful ones. Readers can see how this works from figure 13.

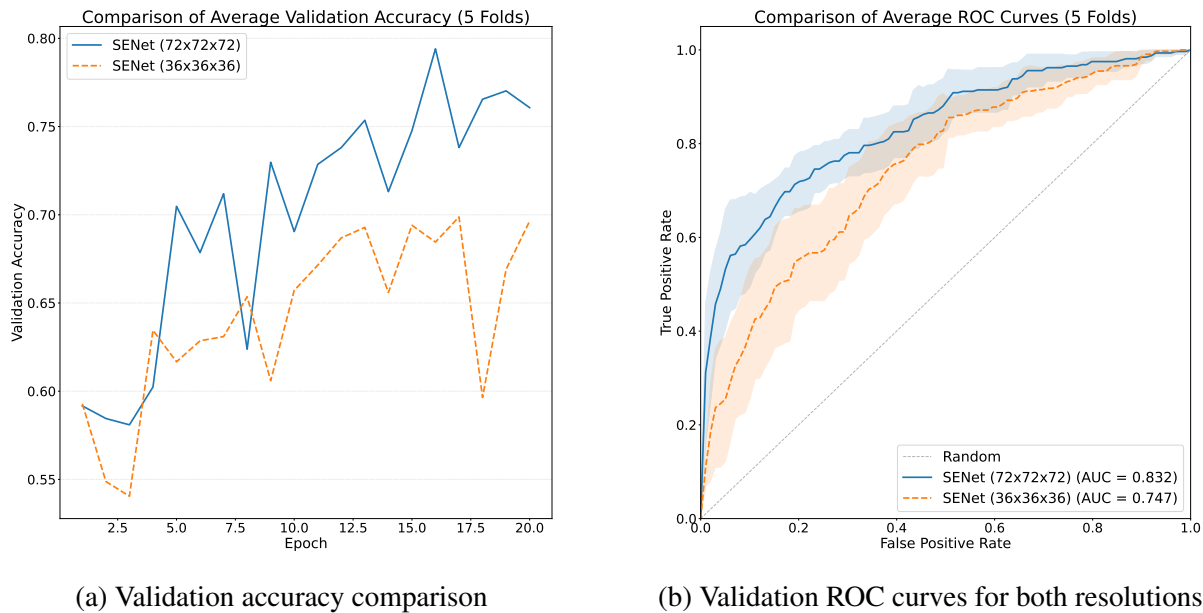


Figure 14: SENet average validation accuracy and ROC curves across 5 folds, showing performance at $72 \times 72 \times 72$ and $36 \times 36 \times 36$ resolutions.

The charts show SENet’s $72 \times 72 \times 72$ model (blue) consistently outperforms the $36 \times 36 \times 36$ model (orange). The accuracy plot shows higher performance (peaking near 80% vs 70%), while the ROC curves demonstrate better classification ability (AUC 0.832 vs 0.747) with tighter confidence bands on the validation set, indicating more reliable predictions across all validation folds.

5.5 Performance Comparison of Models

The performance of the trained model was evaluated with the metrics proposed in section 5.3. The results are shown in Table 6, and shows good results with the baseline ResNet-50. It is also important to note that these metrics were not evaluated with cross validation. Semester Two focused on the training of other models and also using cross validation to get a robust calculation of metrics.

Architecture	Resolution	Accuracy	Precision	Recall	AUC Score
ResNet-50					
ResNet-50	$72 \times 72 \times 72$	76.25	75.63	61.05	79.76
ResNet-50	$36 \times 36 \times 36$	72.64	71.47	55.44	78.44
ResNet-34					
ResNet-34	$72 \times 72 \times 72$	76.11	74.96	64.56	82.29
DenseNet					
DenseNet-201	$72 \times 72 \times 72$	82.22	80.86	72.28	88.61
DenseNet-201	$36 \times 36 \times 36$	71.25	69.60	51.58	74.92
SENet					
SENet	$72 \times 72 \times 72$	80.28	85.13	62.11	86.76
SENet	$36 \times 36 \times 36$	68.19	67.99	46.32	75.59

Table 6: Model Performance Comparisons with different resolutions

Comparative Analysis

Architecture Comparison: DenseNet-201 was the top performer with the highest accuracy (82.22%), recall (72.28%), and AUC score (88.61%) at $72 \times 72 \times 72$ resolution. This suggests that DenseNet's dense connectivity pattern, which enables feature reuse through direct connections between layers, is particularly effective for the complex feature extraction required for 3D medical imaging classification.

Resolution Impact: Across all architectures, models trained on higher resolution inputs ($72 \times 72 \times 72$) consistently outperformed the lower resolution ($36 \times 36 \times 36$) images. The performance gap is more noticeable for DenseNet-201, where accuracy decreased by nearly 11% (82.22% to 71.25%) when using lower resolution images. This indicates that spatial detail preservation is important for medical imaging classification.

Precision-Recall Balance: While SENet achieved the highest precision (85.13%) at high resolution, its recall (62.11%) was notably lower than DenseNet-201's (72.28%). This suggests that SENet excels at minimizing false positives but may miss more positive cases than DenseNet. This trade-off between precision and recall could be significant depending on the application such as a production system.

ResNet Performance: The deeper ResNet-50 marginally outperformed ResNet-34 in accuracy and precision at high resolution, but interestingly, ResNet-34 demonstrated better recall (64.56% vs. 61.05%) and AUC score (82.29% vs. 79.76%). This indicates that the deeper ResNet-50 model was likely overfitting on the training data and did not generalize well to the test set.

Research Questions - Summary of Findings

RQ1: Data Transformations - While the current results don't directly address the specific effects of data transformations such as rotation, scaling, and cropping, they establish a baseline for further experiments, where these transformations would be systematically evaluated against the established baseline metrics.

RQ2: Comparison of Deep Learning Architectures - The results clearly demonstrate that DenseNet-201 outperforms other tested architectures (ResNet-34, ResNet-50, SENet) for Alzheimer's disease classification, achieving the highest accuracy (82.22%), recall (72.28%), and AUC score (88.61%). This suggests that the dense connectivity pattern of DenseNet is particularly well-suited for capturing complex spatial features in 3D MRI scans. Meanwhile, SENet shows specialised capability in achieving high precision (85.13%), indicating potential value in applications where minimising false positives is critical.

RQ3: Impact of Image Resolution - The comparison between high ($72 \times 72 \times 72$) and low ($36 \times 36 \times 36$) resolution inputs reveals consistently better performance with higher resolution across all architectures. The performance degradation is especially pronounced in DenseNet-201 (11% accuracy drop) and SENet (12% accuracy drop), confirming that spatial detail preservation significantly impacts model performance in this domain. Higher resolution inputs maintain critical diagnostic features that are essential for accurate classification, though this comes with increased computational requirements. Memory consumption is considerably higher with the ($72 \times 72 \times 72$) resolution compared to the lower ($36 \times 36 \times 36$). Increasing resolution higher than this comes with a GPU memory bottleneck and more dedicated memory is needed.

Unintentional Data Leakage Issue

During the training of model architectures using cross validation, I observed substantial performance, with accuracy across the 5 different folds consistently approaching or exceeding 90%. Initially, this did not raise concerns, but when a model trained on a lower resolution dataset ($36 \times 36 \times 36$) maintained similarly high performance, it prompted further investigation. In this context, performance should have been considerably lower compared to the higher resolution ($72 \times 72 \times 72$) dataset. This inconsistency suggested a potential issue with the training or testing implementation.

After thoroughly reviewing my implementation and workflow without identifying any obvious errors, insight came during a lecture in my Data Science in Practice module with Brenda Mullally. The discussion emphasised that training and testing data must be completely separate, even when dealing with subsets from the same sources, specifically, patient data. This highlighted my oversight. I had not accounted for patients having multiple scans in my data splitting strategy. Rather than splitting by patients themselves, I had only split by labels. This methodological error resulted in significant data leakage, as some patients had scans in both the training and test sets. Consequently, the architectures learned specific patient brain structures and modularity patterns rather than generalisable features of Alzheimer's Disease, artificially inflating performance metrics as models essentially learned to recognise individual patients and their associated labels.

6 System Deployment

6.1 Application Interface

A prototype application was developed using Node.js and React to demonstrate how the system could be integrated into a professional clinical environment. This application provides several key functionalities:

- User authentication and role-based access control
- Patient profile management (creation, viewing, and updating)
- Medical record integration, including check-up history and medication tracking
- MRI scan uploading and storage
- Integration with the trained deep learning models for automated analysis

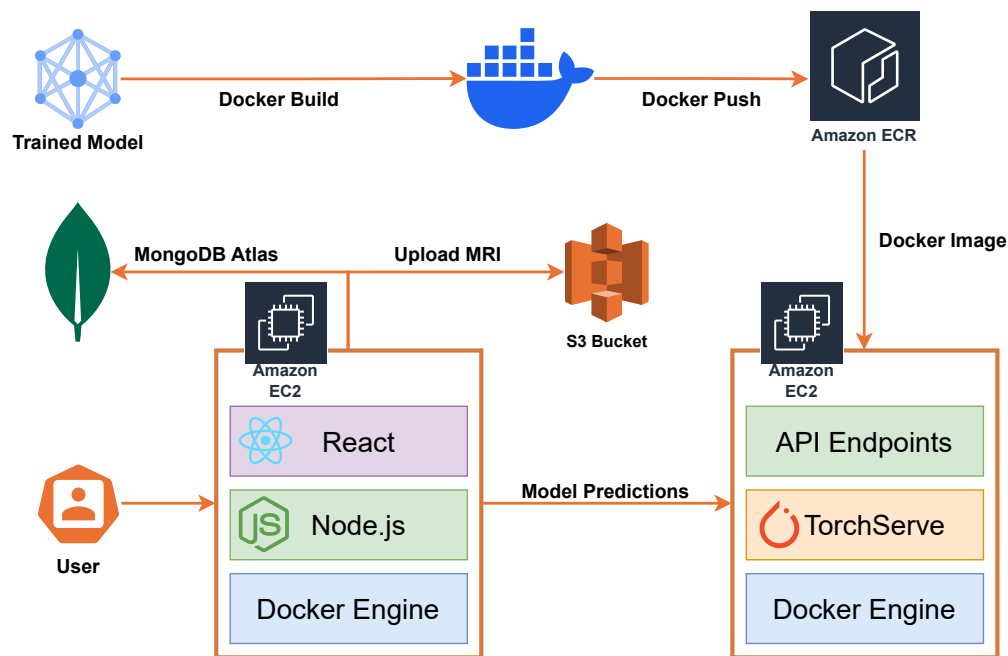


Figure 15: Deployment Platform diagram using AWS.

The application's workflow allows medical professionals to securely log in, manage patient information, upload MRI scans, and receive predictions from the deployed models through a streamlined interface. This enables quick access to AI-assisted diagnostic information while maintaining patient data within a comprehensive medical record system.

6.2 Model Deployment Architecture

For the deployment of trained models, a containerized approach was implemented using Docker in conjunction with TorchServe. This architecture provides several advantages:

- **Encapsulation:** Docker containers package the model and all its dependencies, ensuring consistent operation across different environments
- **Scalability:** The containerised deployment can be easily scaled to handle varying workloads
- **Standardised API:** TorchServe provides an inference API with consistent endpoints for prediction requests
- **Monitoring:** Built-in metrics collection for tracking model performance and utilisation

The deployment pipeline involves compiling the trained PyTorch models into serialized model archives, packaging them within Docker images, and uploading these images to Amazon Elastic Container Registry (ECR). From ECR, the containers can be deployed to various compute resources including Amazon EC2 instances or Kubernetes clusters as needed.

The knowledge and experience gained from my 3rd year module in Automated Cloud Services and the 4th year Semester One module in Agile Software Practice were very helpful in implementing this deployment architecture. The Automated Cloud Services module provided a strong foundation in AWS infrastructure management, load-balancing and auto-scaling. This was applied when setting up the ECR repository, configuring EC2 instances, and setting up the S3 bucket for MRI storage.

7 Tools and Technologies

The tools and technologies can be categorised into two parts which in their own way are interconnected. Part one is the experimentation stage. Initial stages of the data analysis, model experimentation, and model evaluation metrics was done at this phase. Part two focused on application development and deployment. This will connect the model to the application so end users can interact with the trained model.

7.1 Part One - Experimentation

Part one is comprised of data exploratory analysis, data preprocessing and model experimentation and development. A range of tools will be used for getting the dataset in a form that's ready for modelling and also evaluating models performance.

Python

Python is the programming language that will be used for the Data Analytics, Data Exploratory Analysis, and Model Development. Python is an interpreted language which means code is run through the interpreter for execution rather than compiling the code to machine code. It is a high level, object-oriented language and allows for rapid development because of it's ease of use.

PyTorch

PyTorch is a fully featured Python based framework for building and deploying deep neural networks. Deep neural networks are an off branch of machine learning that is commonly used in applications like image recognition and natural language processing (NLP). PyTorch also has intuitive support for Graphical Processing Units (GPUs), more precisely, support for NVIDIA's (CUDA) platform. This will offload the heavy tasks to the GPU such as training deep models which will increase training times.

Jupyter

Jupyter is a web based interactive development environment. The idea behind Jupyter is that you work within a notebook, which allows you to write and execute code in a cell by cell manner. Cells can contain text, code, and even markdown. Jupyter notebooks are useful when you want to demonstrate your code or keep track of experiments in a tidy manner.

Medical open network for AI (MONAI)

MONAI [20] is a consortium-led PyTorch-based framework for deep learning in healthcare. MONAI extends PyTorch to support medical data, with a particular focus on imaging, and provide purpose-specific AI model architectures, transformations and utilities that streamline the development of medical AI models.

FSL

FSL [15] is a comprehensive library of analysis tools for FMRI, MRI, and diffusion brain imaging data. For this project, the FSL toolkit provides a range of useful tools for preprocessing MRI and FMRI image data. Two tools from the FSL application will be used: the first is FLIRT (FMRIB's Linear Image Registration Tool), a fully automated, robust, and accurate tool for linear (affine) intra- and inter-modal brain image registration. The second tool is BET (Brain Extraction Tool), which removes non-brain tissue from whole-head images. BET can also estimate the inner and outer skull surfaces, as well as the outer scalp surface, provided that high-quality T1 and T2 images are available.

7.2 Part Two - Deployment

Part two focused on comparing other model architectures and the usability of the model, users can upload an image to the model for inference and processing. Below is a list of tools that was utilised for the deployment and the development of the system pipeline.

React

React.js is a JavaScript library for building efficient and flexible fronted applications. Its component architecture allows developers to create UI elements such as forms, tables, and navigation bars with usability in mind. With React's state management you also get the flexibility of manipulating and updating UI components with immediate effect to the user.

Node.js

Node.js is an open-source, cross-platform JavaScript runtime environment and library for running web applications outside the client's browser. It provides an event-driven, non-blocking I/O model, which makes it highly efficient and scalable. This will enable the frontend (React) to connect to the backend (database and PyTorch model API endpoint).

TorchServe

TorchServe is a tool that works with PyTorch that enables you to deploy trained models at scale. With its ease of use, and extensive documentation, getting a model deployed can be integrated into the production pipeline with little effort.

Streamlit

Streamlit lets you transform Python scripts into interactive web applications for the user to interact with your trained models. Streamlit is built with Python, which enables fast and interactive prototyping for data driven web applications.

8 Project Plan

This section outlines the overall project structure, goals, and methodologies that was followed throughout the duration of the project. The project is divided into two main phases, Semester One and Semester Two. The project plan also discusses challenges faced during semester one and also potential challenges in semester two. Gantt charts was also created at the start of semester one, the reasoning behind it was to get a better understanding of what sections of the report along with the project plan needed to be completed with time constraints, this has a sense of urgency for certain sections of both the project report and also the project research and development.

8.1 Semester One

Semester One focused on building an experimentation and software framework to carry out the development and testing of models in Semester Two. Furthermore, researching existing literature related to the project's domain in order to explore relevant techniques and models that could be applied to this project. This includes reviewing current work in similar areas and also studying methodologies, models used, and common challenges encountered in similar projects and research. This initial phase aims to provide insights that will help with model selection and also with data preprocessing and transformations.

The dataset used in this project was obtained on May 2nd, 2024, following an initial access request submitted in January 2024. The request included the project's scope and intended use of the dataset. Access to this dataset was crucial for the feasibility of the project. While other datasets, such as the Open Access Series of Imaging Studies (OASIS), are available, they were not necessary as access to the ADNI dataset mentioned in section 4 was granted.

8.1.1 Gantt Chart

The Gantt chart below (Figure 16) illustrates the week-by-week breakdown of tasks for Semester One, highlighting the workload distribution and achieved project milestones.

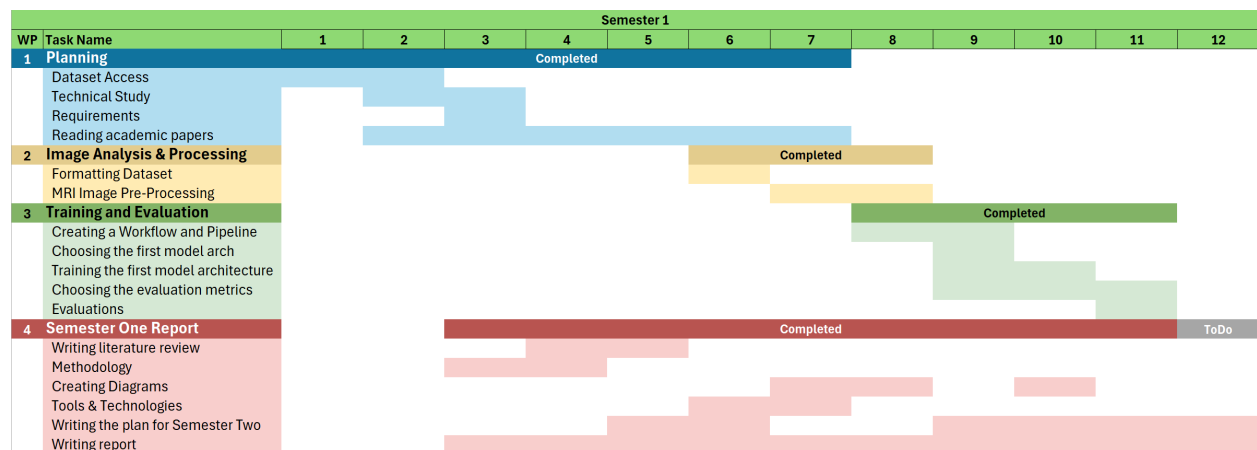


Figure 16: Workload schedule for Semester One, showing weekly task allocations and milestones.

As illustrated in Figure 16, the chart outlines the key work packages and the associated tasks that were necessary to complete each package. Milestones reached during this semester are clearly marked as "completed", showcasing the progress made. This demonstrates that the project has not only aligned to the planned schedule but has also met the objectives for Semester One.

The chart also shows the approach undertaken in managing the project, with tasks sequenced to achieve steady progress. Each milestone reached reflects significant steps toward the overall project goals, including data preparation, model evaluation, and initial experimentation. This progress indicates that the project is well aligned with the semester timeline, and sets a strong foundation for the upcoming work in Semester Two, which will focus more on the comparison of different model architectures and transformations with respect to model performance. Additionally, a react based web application will be designed and developed to allow users to create patient profiles, upload MRI scans, and use the cloud deployed models for predictions.

8.2 Semester Two

In Semester Two, the primary focus was on the full development and evaluation of the models. This involves training more models, mentioned in section 5.4 and evaluating them based on the methods outlined in section 5.3. Following the model evaluations, the next part will be to build a demo application for the models use in a production environment in a health care setting. This will be revolved around a Node.js and React based web application which will allow professionals to create an account, create patient profiles, and upload patient related information. Lastly, the system will also allow the user to upload MRI scans of the patient and use the trained models to make deep learning assisted predictions on the medical images.

8.2.1 Gantt Chart

The Gantt chart below (Figure 17) illustrates the week-by-week breakdown of tasks for Semester Two, highlighting the workload distribution and project milestones that have been achieved such as selecting models to train and also deploying onto AWS using TorchServe.

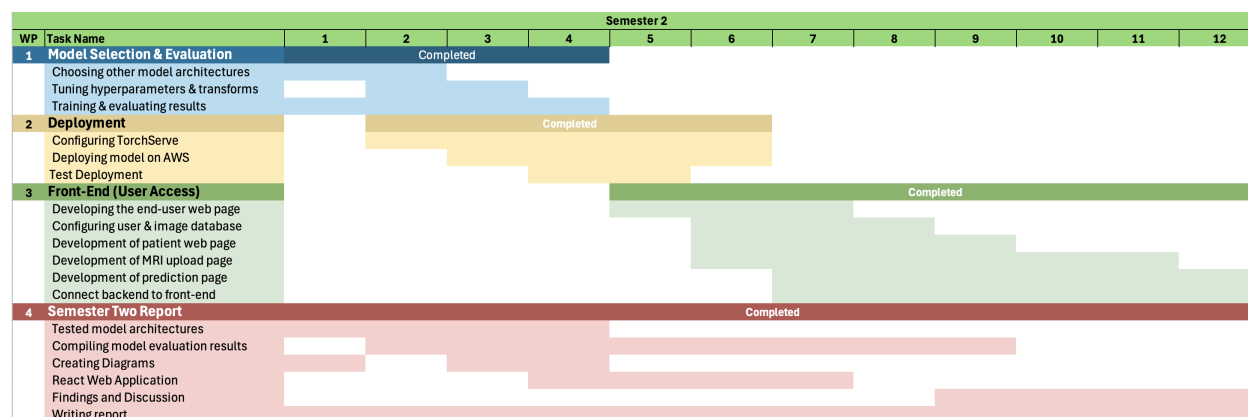


Figure 17: Workload schedule for Semester Two, showing weekly task allocations and milestones.

To get a better understanding of how the web application might function, wireframes provide a visual representation of its potential layout and structure. These wireframes help outline the design and organisation of key elements such as navigation, user inputs, and how predictions from the deployed model will be presented to the user.

The wireframes developed are intentionally kept simple to serve as a foundational guide for the application's development. These designs represent early stage concepts of potential layouts but play an important role in visualising the user interfaces for the project. The full realisation of the design can be seen in section 9 in which screenshots of the Cognify platform are shown.

8.2.2 Homepage Wireframe



Figure 18: Homepage wireframe for the proposed web application.

8.2.3 Patient Page Wireframe

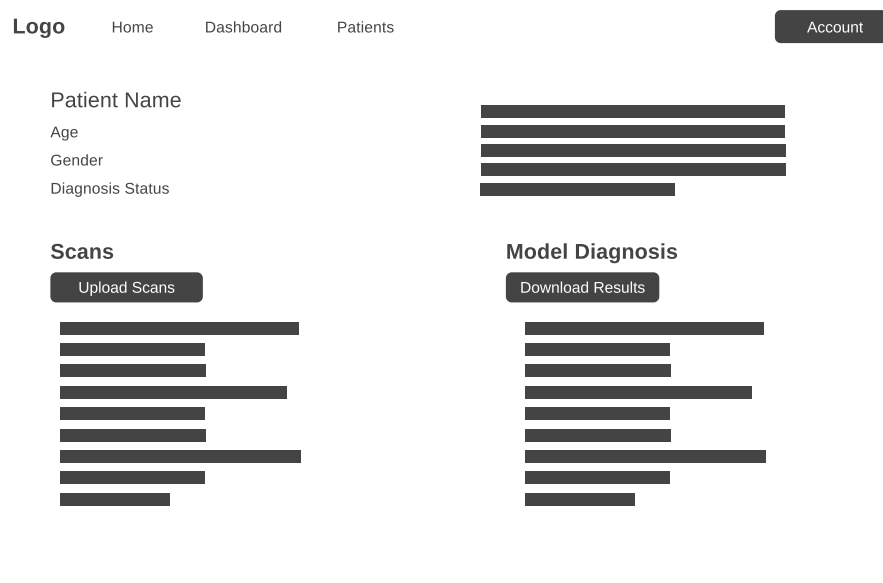


Figure 19: Patient page wireframe for the proposed web application.

8.2.4 Personas and Use Cases



Dr. Emily Carter

Role: Neurologist

Speciality: Alzheimer's Disease

Experience:
12 years specialising in neurological disorders, with a focus on Alzheimer's disease.

Emily's Use Cases

- Upload MRI scans for patients suspected of having Alzheimer's.
- Analyse results using the AI model to identify signs of Alzheimer's disease.
- Add and review patient notes for ongoing diagnosis and treatment planning.
- Access historical patient records to monitor disease progression over time.

Emily's Features

- Intuitive dashboard with patient profiles.
- Easy upload and integration of MRI scans.
- Quick access to model-generated insights with predictions for each class.
- Option to print or share reports with other healthcare colleagues.

Figure 20: User persona of Dr. Carter who specialises in Alzheimer's Disease and progression.



Sarah Foley

Role: Radiology Technician

Speciality: Medical Imaging

Experience:
6 years of experience as a Radiology Technician, specialising in medical imaging procedures such as MRI, CT, and X-ray scans.

Sarah's Use Cases

- Upload MRI scans to the system after imaging sessions.
- Check scan quality to ensure they meet the requirements for analysis.
- Upload patient data, including MRI scans and ongoing notes, to ensure experienced physicians have all necessary information to conduct their analysis.

Sarah's Features

- Simplified workflow for uploading and tagging MRI scans.
- Notifications for successful uploads or errors in scan quality.
- Role-based access to ensure only technical tasks are accessible.
- Enable the assignment of doctors and physicians to patient profiles in the database, granting them appropriate access to patient data.

Figure 21: User persona of Ms. Sarah Foley who specialises in Medical Imaging.



Joe Blogs

Role: Patient

Speciality: Patient Under Observation

Experience:
Joe Blogs has been living with Alzheimer's for 3 years, managing symptoms and following a treatment plan provided by his physician.

Joe's Use Cases

- View MRI scan results with simplified visuals or summaries provided by the doctor.
- Read doctors' notes or treatment plans shared with him.
- Download personal medical reports for reference or sharing with family members.

Joe's Features

- Limited, read-only access to personal medical information.
- Simplified interface with no technical jargon.
- Secure login to ensure privacy.

Figure 22: User persona of Mr. Joe Blogs with ongoing progression of Alzheimer's Disease.

9 Web Application Implementation

Following the wireframe designs and personas outlined above, the web application has been developed using Node.js and React as planned. The application serves as a functional demonstration and minimum viable product (MVP), of how the trained deep learning models can be integrated into a healthcare setting for practical use by medical professionals.

The web application features an authentication system using JSON Web Tokens (JWT), patient management capabilities, and most importantly, an interface for uploading and analysing MRI scans using the deployed models. Below are screenshots of the key pages implemented based on the initial wireframes.

9.1 Homepage

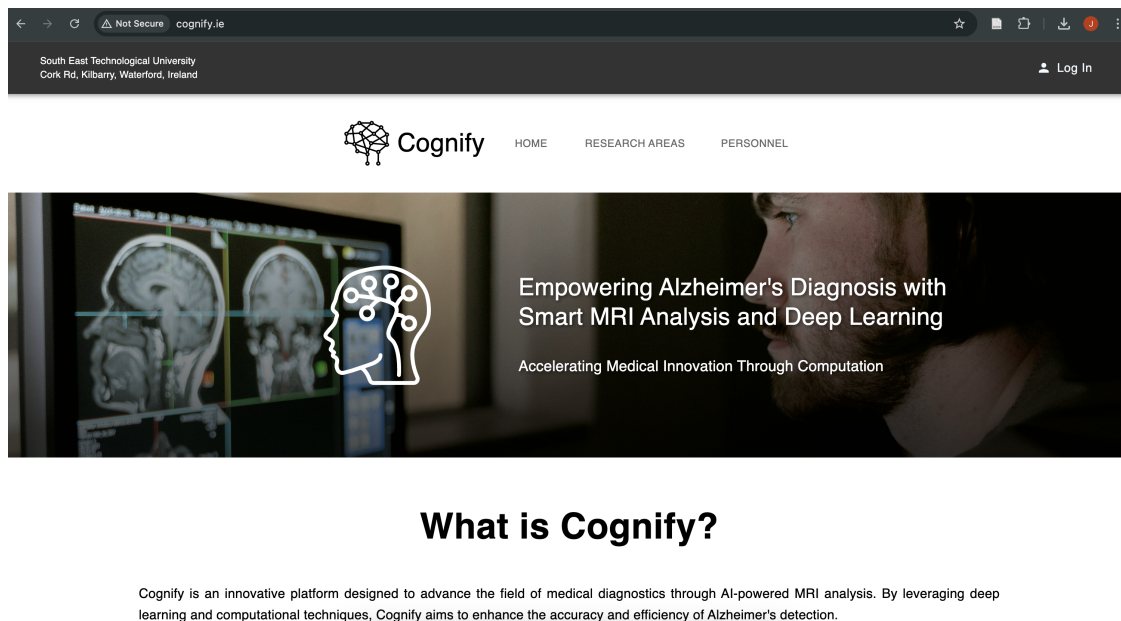


Figure 23: Homepage of web application.

The homepage introduces what Cognify is, and also who is involved in the project which would be Bernard and I.

9.2 Authentication Page

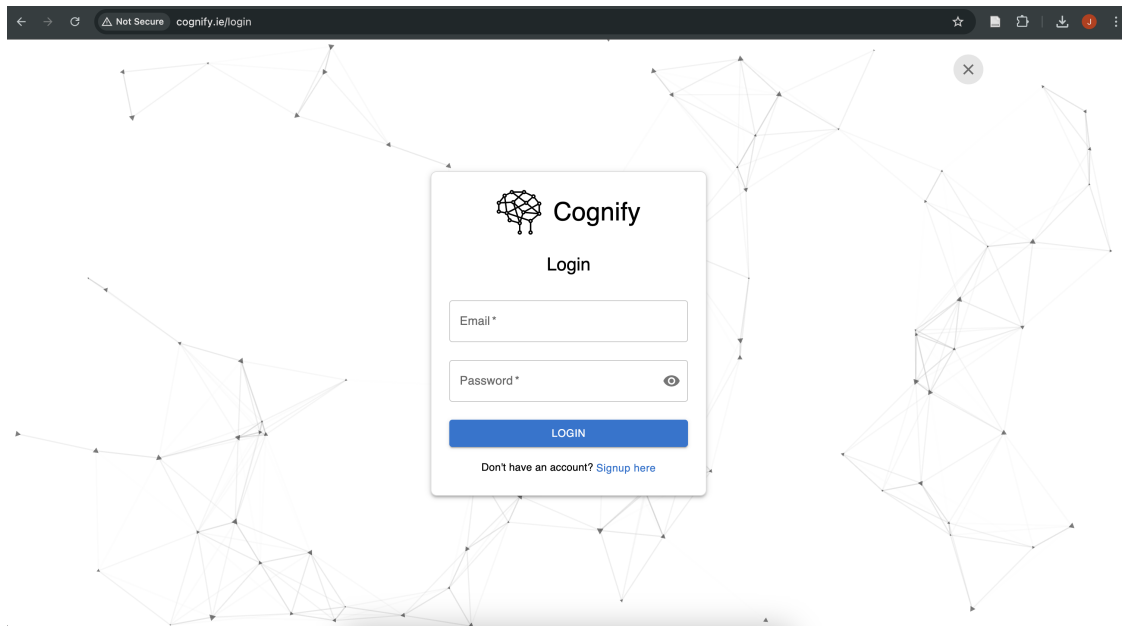


Figure 24: Authentication page for the application.

9.3 Patients Page

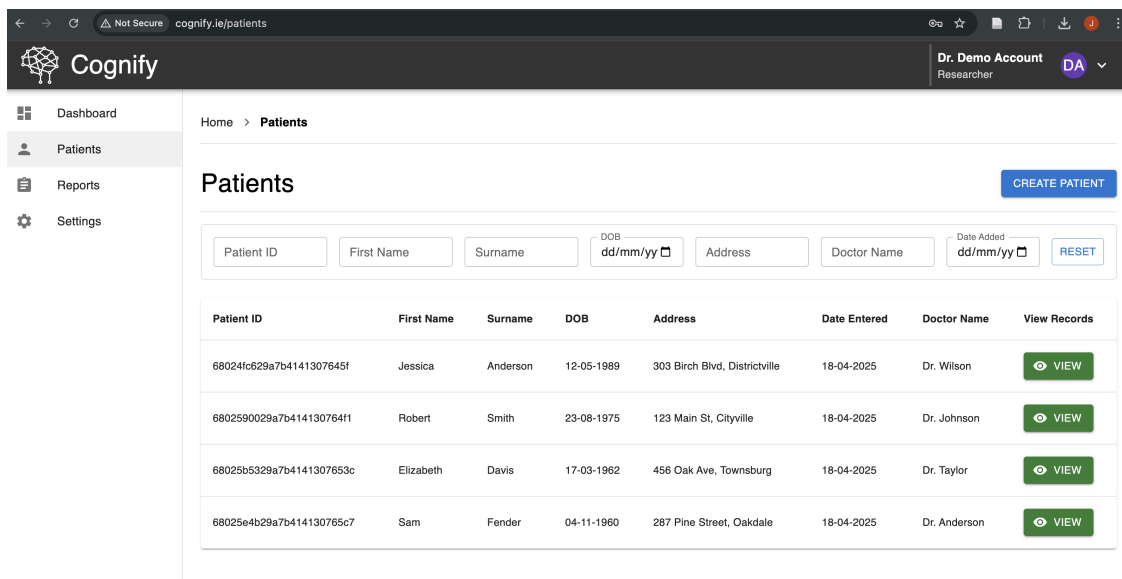


Figure 25: List of patients on the Cognify platform.

9.4 Patient Inference Page

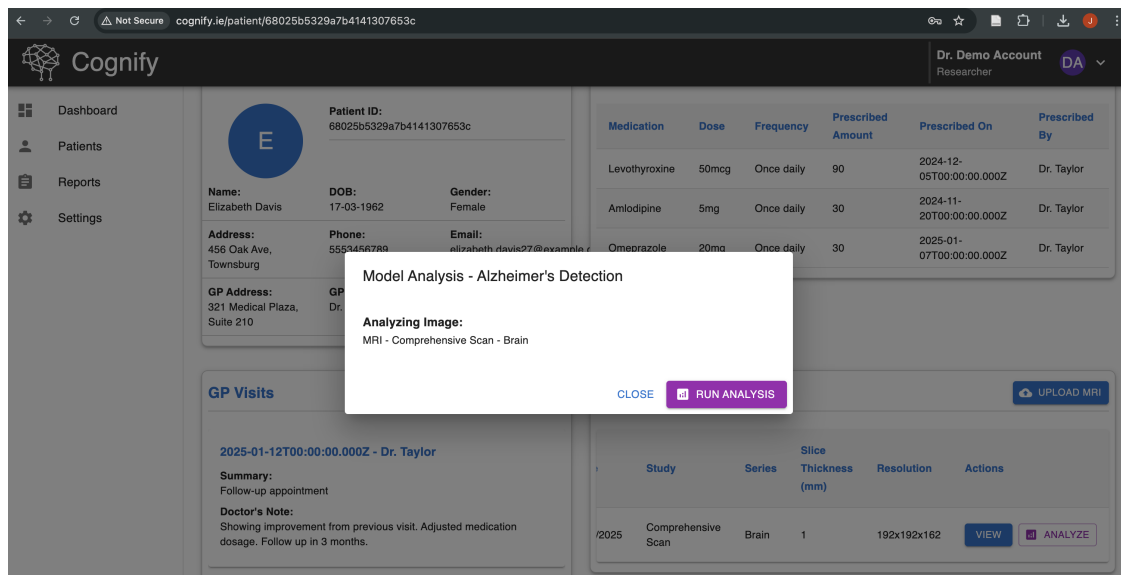


Figure 26: Model Inference on the Cognify platform.

9.5 Model Results

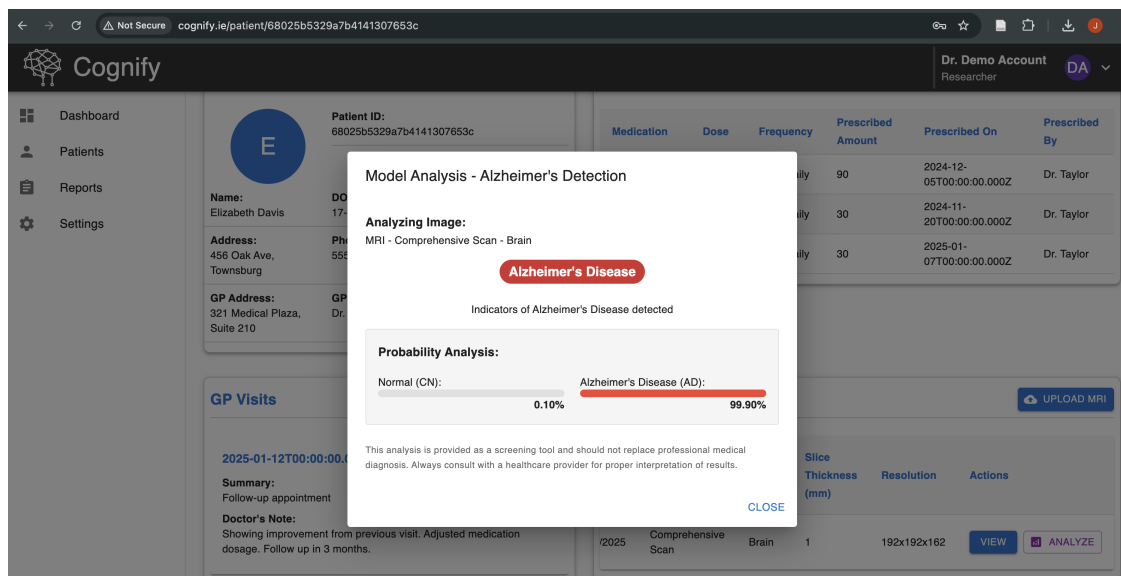


Figure 27: Model Results from the deployed model.

9.6 Challenges and Mitigation Strategies

As part of my research into tooling and frameworks, setting up and configuring the required software posed several challenges to ensure it met project needs. Some tools, such as the FSL library for image preprocessing, required a Linux-based system. To avoid using a completely separate system, I opted to use the Windows Subsystem for Linux (WSL) with a Ubuntu image, which allowed easier integration without the need for dual-booting or virtualisation.

However, I encountered a significant hurdle with storage. The original dataset was over 85GB, and my system had only approximately 50GB of free space available. Reinstalling Windows and the software stack to a larger drive was not a feasible option due to time constraints and also breaking the workflow.

To overcome this, I utilised an SSD cloning tool to transfer the original boot drive to a larger SSD, effectively increasing available storage without changing the software environment. This provided the necessary capacity to handle the dataset and ensured smooth preprocessing for subsequent steps in the project.

Another challenge that I faced, was using MONAI for the first time, although I have never used it before, it builds on top of PyTorch's development best practices. MONAI also has detailed documentation and some tutorials on getting started. Because of this, initial development progressed slowly; being aware of these challenges helped me stay focused and adapt as I worked through the project plan and development process.

In the second semester of the project, when it came to training the models I needed a unique setup. Training the models was going to be computationally expensive which will effect my ability to use my personal desktop to do other college work. The workaround I found very helpful was the use of a Jupyter Lab server. This allowed me to have my desktop running as a server, allowing me to connect remotely from my laptop while I'm in college, do training runs and analysis tasks, while these high resource tasks are running, I can then freely use my laptop to dedicate time to other modules and work.

10 Conclusion and Future Work

This project developed and evaluated different deep learning architectures for classification of Alzheimer's Disease (AD) using a comprehensive deep learning framework in section 5.2 using 3D MRI imaging from the Alzheimer's Disease Neuroimaging Initiative (ADNI) dataset in section 4.

The comparison of deep learning architectures shows that DenseNet-201 had superior performance with the highest accuracy (82.22%), recall (72.28%), and AUC score (88.61%) with a resolution voxel size of $(72 \times 72 \times 72)$. This suggests that DenseNet's dense connectivity pattern, which enables feature reuse through direct connections between layers, is effective for extracting the complex spatial features necessary for 3D medical image classification. Image resolution also proved to be a factor affecting model performance across all architectures, with higher resolution inputs $(72 \times 72 \times 72)$ outperforming lower resolution $(36 \times 36 \times 36)$. This finding outlines the importance of spatial detail preservation in medical imaging classification, though the increased computational demands of higher resolution must be considered in practical applications.

After implementing a comprehensive workflow for training and comparing model architectures, cloud deployment in section 6, was the next phase. With use of TorchServe and docker, the best performing model DenseNet-201 in section 5.4.2, was deployed onto AWS in which it provided an easy access inference API for the web application also deployed in AWS. This cloud system allows the users to then create patient profiles and make inference calls on uploaded MRI images. This platform shows the practicality of a real use case within a health care system for Alzheimer's Disease patients, and also how physicians and doctors could interact and use the system.

Future Work

An area for further exploration would be localising areas within the images that have the biggest effect on the outcome of classifications. For example, given a prediction, what part of the image was the biggest predictor for that outcome? This could be done by creating an activation map of a select few layers within the model, this will create a heat map which could then be mapped to the input image as an overlay. In theory this could create the localised points to show what areas had the highest predictive power for the models output.

This approach, often referred to as Class Activation Mapping (CAM) or Gradient-weighted Class Activation Mapping (Grad-CAM) [21], would provide valuable interpretability for the deep learning models. In the context of medical imaging, this visualisation technique could:

1. Provide clinicians with more transparent AI-assisted diagnoses by highlighting the specific regions that influenced the model's decision.
2. Help validate whether the model is focusing on clinically relevant features rather than incidental patterns or artifacts in the data.
3. Potentially identify novel biomarkers or imaging characteristics that might not be obvious to human observers.
4. Serve as a quality control measure to ensure the model is making predictions based on appropriate anatomical structures.

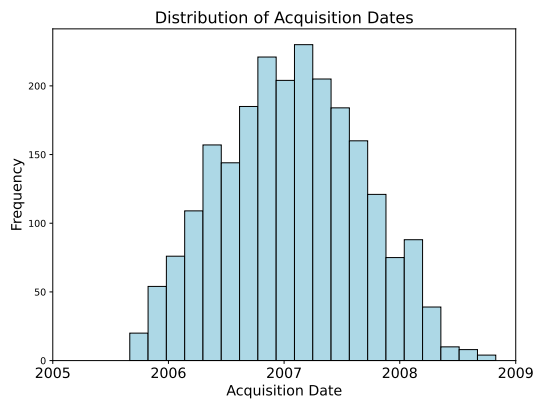
11 Bibliography

- [1] Michael A. DeTure and Dennis W. Dickson. "The neuropathological diagnosis of Alzheimer's disease". In: *Molecular Neurodegeneration* 14.1 (Aug. 2019), p. 32. ISSN: 1750-1326. DOI: 10.1186/s13024-019-0333-5. URL: <https://doi.org/10.1186/s13024-019-0333-5> (visited on 11/04/2024).
- [2] A. P. Porsteinsson et al. "Diagnosis of Early Alzheimer's Disease: Clinical Practice in 2021". en. In: *The Journal of Prevention of Alzheimer's Disease* 8.3 (July 2021), pp. 371–386. ISSN: 2426-0266. DOI: 10.14283/jpad.2021.23. URL: <https://doi.org/10.14283/jpad.2021.23> (visited on 11/04/2024).
- [3] Kaiming He et al. *Deep Residual Learning for Image Recognition*. arXiv:1512.03385. Dec. 2015. DOI: 10.48550/arXiv.1512.03385. URL: <http://arxiv.org/abs/1512.03385> (visited on 11/04/2024).
- [4] eHealth Ireland. *eHealth Strategy for Ireland*. 2023. URL: <https://assets.gov.ie/16174/092e7c62f97b472b83cdb6dfdcdf5c7.pdf> (visited on 06/11/2024).
- [5] ESRI et al. *Developments in healthcare information systems in Ireland and internationally*. en. Tech. rep. ESRI, June 2021. DOI: 10.26504/sustat105. URL: <https://www.esri.ie/publications/developments-in-healthcare-information-systems-in-ireland-and-internationally> (visited on 11/06/2024).
- [6] Bruno Dubois et al. "Timely Diagnosis for Alzheimer's Disease: A Literature Review on Benefits and Challenges". en. In: *Journal of Alzheimer's Disease* 49.3 (Dec. 2015). Ed. by Andrew Saykin, pp. 617–631. ISSN: 13872877, 18758908. DOI: 10.3233/JAD-150692. URL: <https://www.medra.org/servlet/aliasResolver?alias=iospress&doi=10.3233/JAD-150692> (visited on 11/06/2024).
- [7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. "ImageNet classification with deep convolutional neural networks". en. In: *Communications of the ACM* 60.6 (May 2017), pp. 84–90. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/3065386. URL: <https://dl.acm.org/doi/10.1145/3065386> (visited on 11/19/2024).
- [8] Mana Saleh Al Reshan et al. "Detection of Pneumonia from Chest X-ray Images Utilizing MobileNet Model". en. In: *Healthcare* 11.11 (Jan. 2023). Number: 11 Publisher: Multidisciplinary Digital Publishing Institute, p. 1561. ISSN: 2227-9032. DOI: 10.3390/healthcare11111561. URL: <https://www.mdpi.com/2227-9032/11/11/1561> (visited on 10/10/2024).
- [9] Lulu Gai et al. "Comparing CNN-based and transformer-based models for identifying lung cancer: which is more effective?" en. In: *Multimedia Tools and Applications* 83.20 (June 2024), pp. 59253–59269. ISSN: 1573-7721. DOI: 10.1007/s11042-023-17644-4. URL: <https://doi.org/10.1007/s11042-023-17644-4> (visited on 11/15/2024).
- [10] Hugo Touvron et al. *Training data-efficient image transformers & distillation through attention*. arXiv:2012.12877. Jan. 2021. DOI: 10.48550/arXiv.2012.12877. URL: <http://arxiv.org/abs/2012.12877> (visited on 11/17/2024).
- [11] Ananya Jain et al. *A Comparative Study of CNN, ResNet, and Vision Transformers for Multi-Classification of Chest Diseases*. arXiv:2406.00237. May 2024. URL: <http://arxiv.org/abs/2406.00237> (visited on 11/14/2024).
- [12] Prajit Sengupta, Anant Mehta, and Prashant Singh Rana. "Enhancing Performance of Deep Learning Models with a Novel Data Augmentation Approach". In: *2023 14th International*

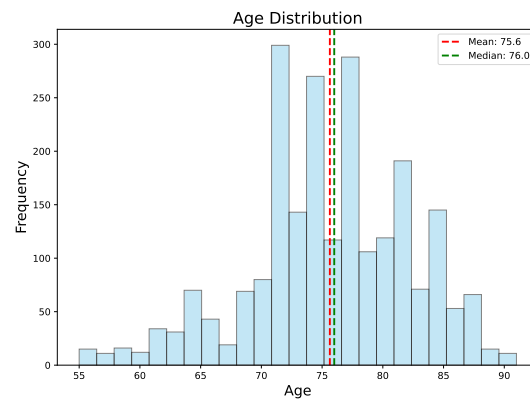
- Conference on Computing Communication and Networking Technologies (ICCCNT)*. ISSN: 2473-7674. July 2023, pp. 1–7. DOI: 10.1109/ICCCNT56998.2023.10308298. URL: <https://ieeexplore.ieee.org/document/10308298/?arnumber=10308298> (visited on 11/27/2024).
- [13] Xinle Gao, Zhiyong Xiao, and Zhaohong Deng. “High accuracy food image classification via vision transformer with data augmentation and feature augmentation”. en. In: *Journal of Food Engineering* 365 (Mar. 2024), p. 111833. ISSN: 02608774. DOI: 10.1016/j.jfoodeng.2023.111833. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0260877423004314> (visited on 11/27/2024).
- [14] R C. Petersen et al. “Alzheimer’s Disease Neuroimaging Initiative (ADNI)”. In: *Neurology* 74.3 (Jan. 2010), pp. 201–209. ISSN: 0028-3878. DOI: 10.1212/WNL.0b013e3181cb3e25. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC2809036/> (visited on 10/11/2024).
- [15] Mark Jenkinson et al. “FSL”. eng. In: *NeuroImage* 62.2 (Aug. 2012), pp. 782–790. ISSN: 1095-9572. DOI: 10.1016/j.neuroimage.2011.09.015.
- [16] William R. Crum et al. “Registration of challenging pre-clinical brain images”. en. In: *Journal of Neuroscience Methods* 216.1 (May 2013), p. 62. DOI: 10.1016/j.jneumeth.2013.03.015. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC3683149/> (visited on 10/23/2024).
- [17] Vs Fonov et al. “Unbiased nonlinear average age-appropriate brain templates from birth to adulthood”. en. In: *NeuroImage* 47 (July 2009), S102. ISSN: 10538119. DOI: 10.1016/S1053-8119(09)70884-5. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1053811909708845> (visited on 03/04/2025).
- [18] Alexander Selvikvåg Lundervold and Arvid Lundervold. “An overview of deep learning in medical imaging focusing on MRI”. In: *Zeitschrift für Medizinische Physik*. Special Issue: Deep Learning in Medical Physics 29.2 (May 2019), pp. 102–127. ISSN: 0939-3889. DOI: 10.1016/j.zemedi.2018.11.002. URL: <https://www.sciencedirect.com/science/article/pii/S0939388918301181> (visited on 11/06/2024).
- [19] Jie Hu et al. *Squeeze-and-Excitation Networks*. arXiv:1709.01507 [cs] version: 4. May 2019. DOI: 10.48550/arXiv.1709.01507. URL: <http://arxiv.org/abs/1709.01507> (visited on 04/02/2025).
- [20] M. Jorge Cardoso et al. *MONAI: An open-source framework for deep learning in healthcare*. arXiv:2211.02701. Nov. 2022. URL: <http://arxiv.org/abs/2211.02701> (visited on 10/11/2024).
- [21] Ramprasaath R. Selvaraju et al. “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization”. In: *International Journal of Computer Vision* 128.2 (Feb. 2020). arXiv:1610.02391 [cs], pp. 336–359. ISSN: 0920-5691, 1573-1405. DOI: 10.1007/s11263-019-01228-7. URL: <http://arxiv.org/abs/1610.02391> (visited on 04/09/2025).

A Trained Model Figures

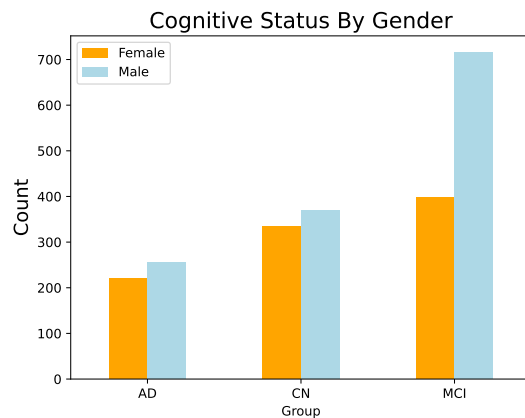
Initial EDA Analysis:



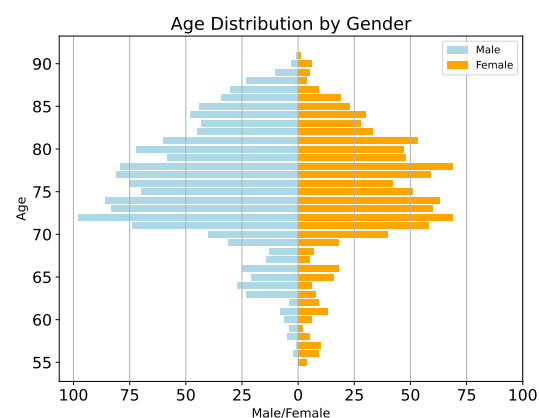
(a) Acquisition Dates



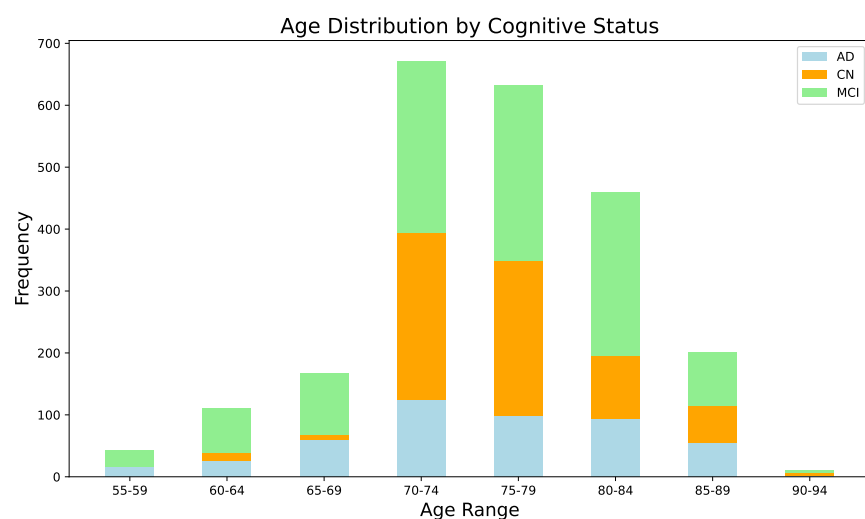
(b) Age Distribution



(c) Cognitive Status By Gender



(d) Age Distribution By Gender

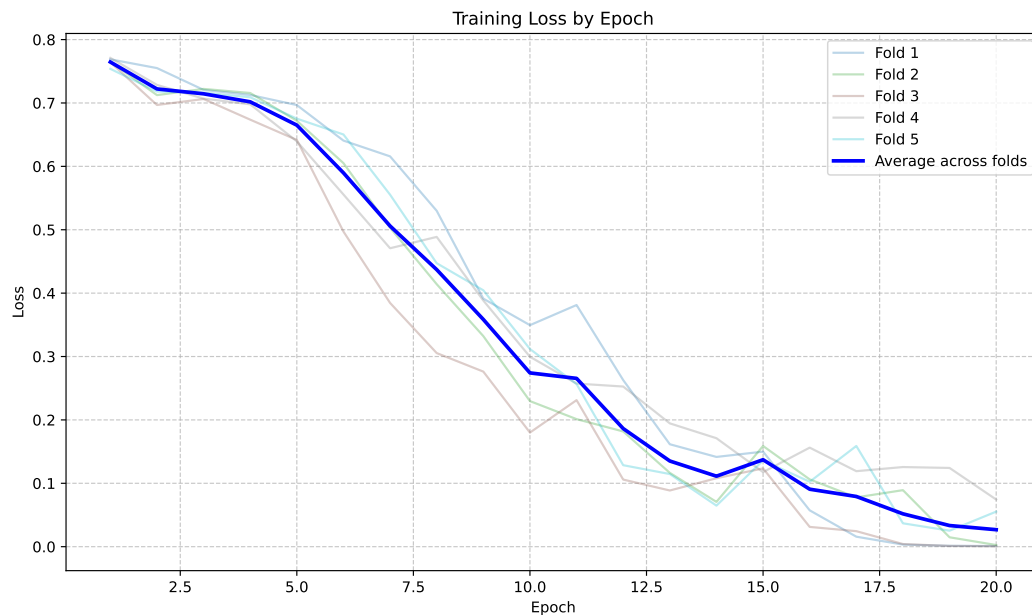


(e) Age Distribution by Cognitive Status

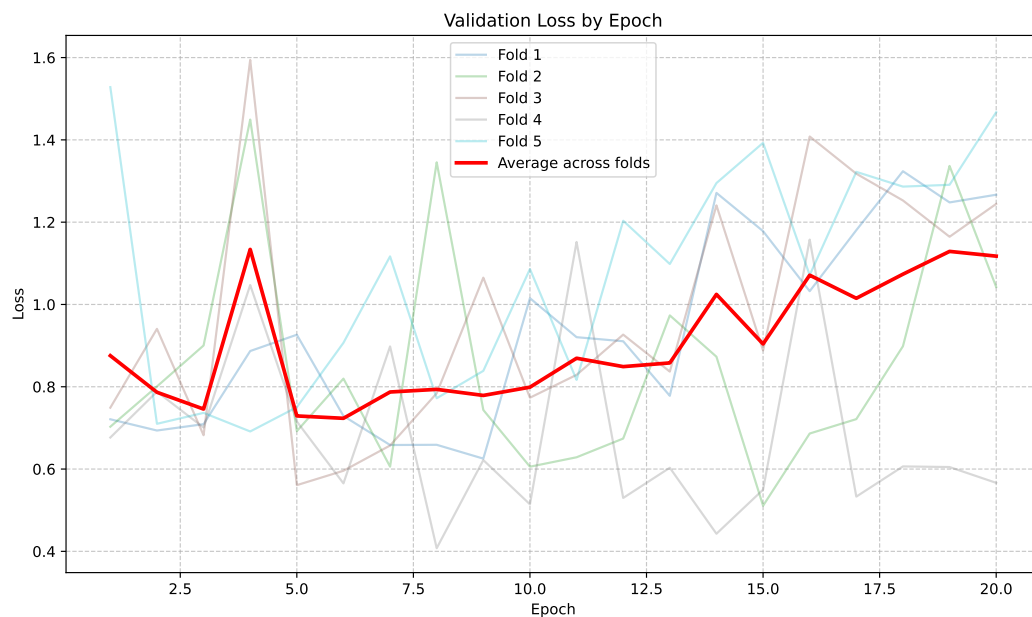
Figure 28: Initial EDA Analysis Figures.

A.1 ResNet-50 72x72x72 Graphs

Figure 29 below show the training and validation loss curves for each of the 5 folds for cross validation.



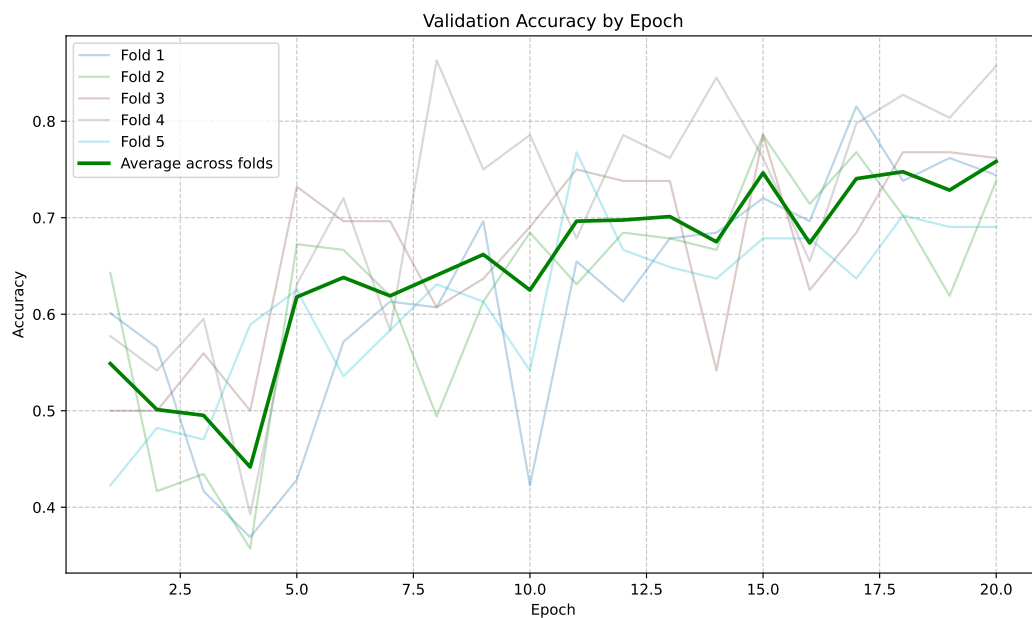
(a) Training loss average and each of the 5 folds



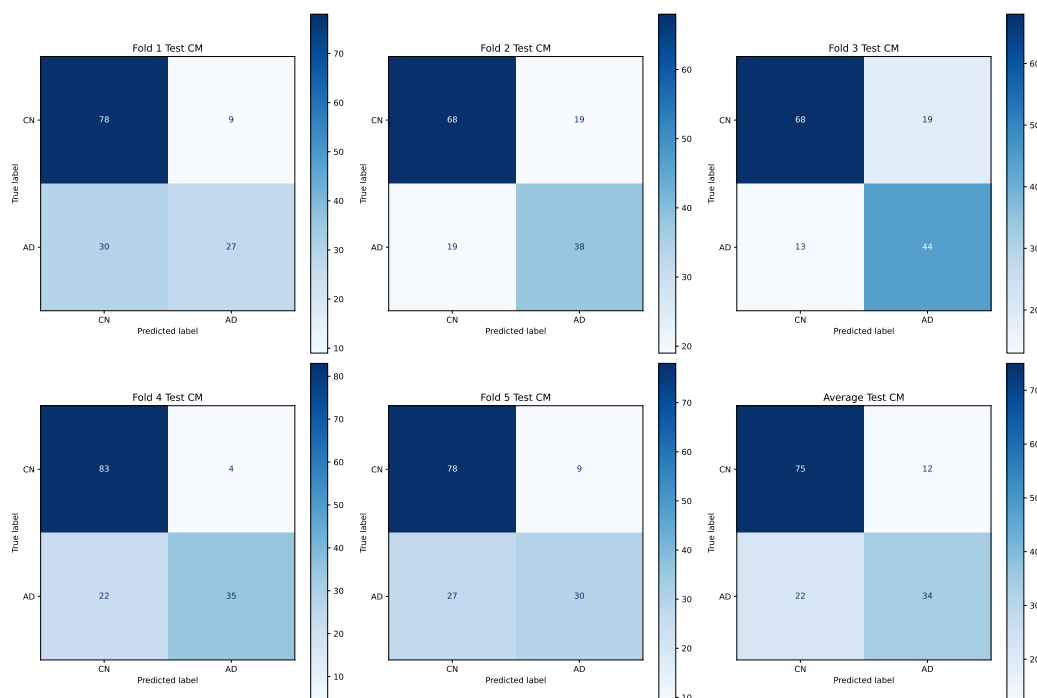
(b) Validation loss average and each of the 5 folds

Figure 29: ResNet-50 72 Training Performance Metrics (Part 1)

Figure 30 (a) also shows the validation accuracy for each of the 5 folds and figure 30 (b) shows the confusion matrix for each of the 5 folds along with the average (test) metrics on the bottom right corner.



(a) Validation accuracy average and each of the 5 folds

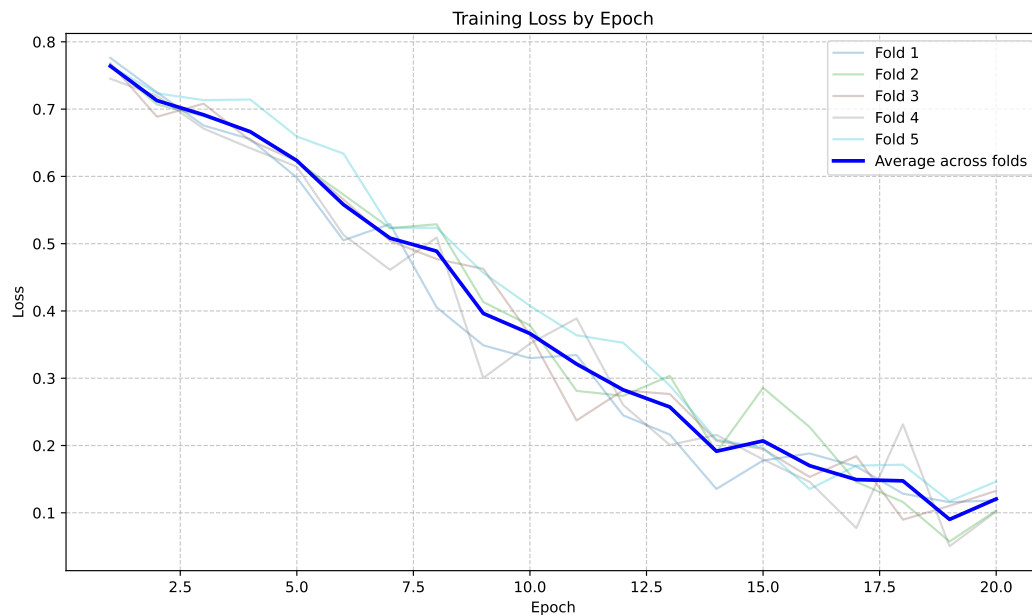


(b) Confusion Matrix for each of the 5 folds and the test set matrix

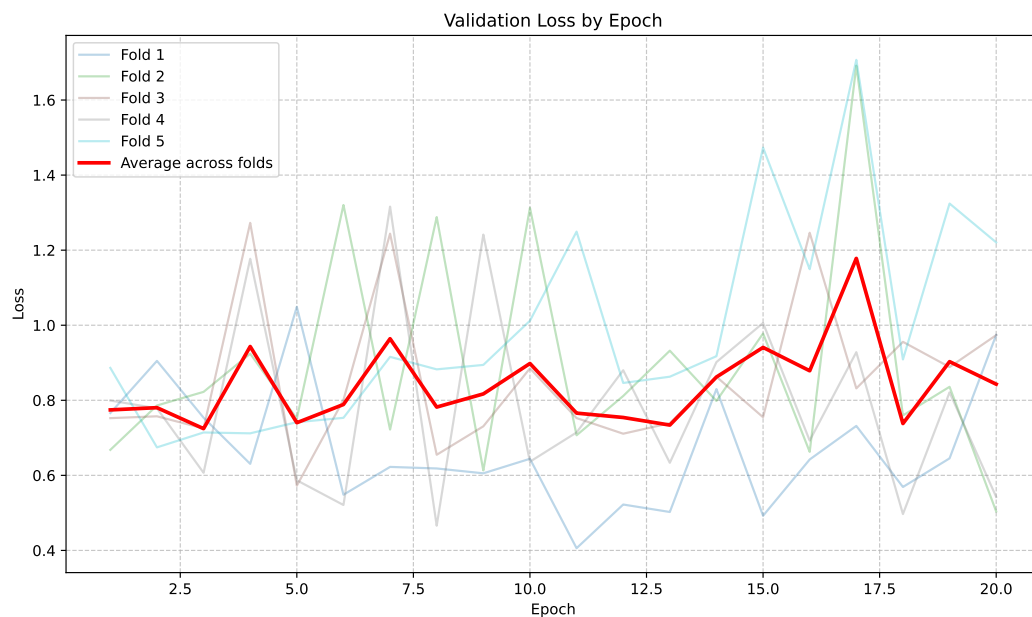
Figure 30: ResNet-50 Training Performance Metrics (Part 2)

A.2 ResNet-50 36x36x36 Graphs

Figure 29 below show the training and validation loss curves for each of the 5 folds for cross validation.



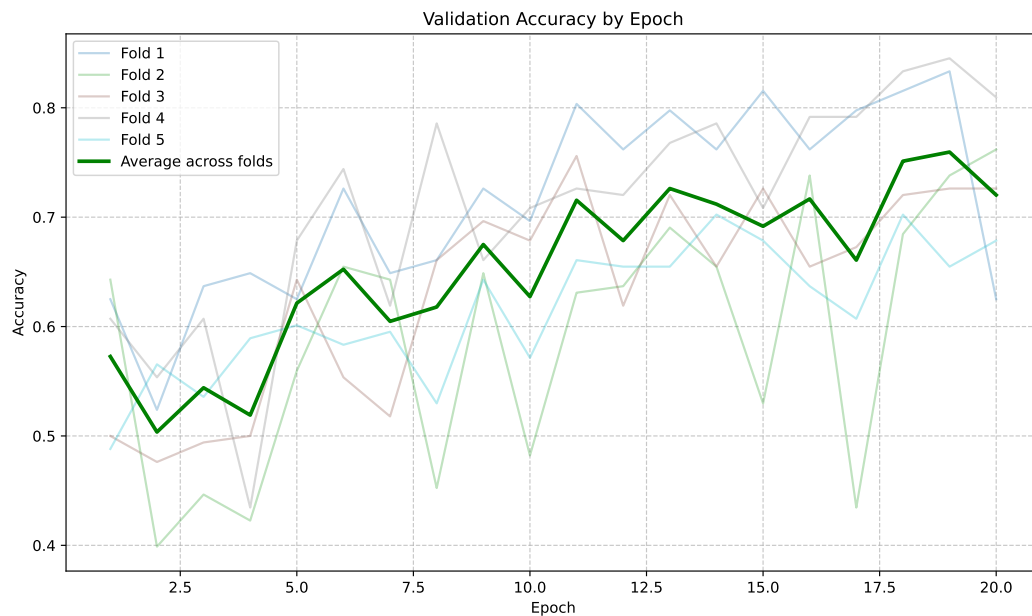
(a) Training loss average and each of the 5 folds



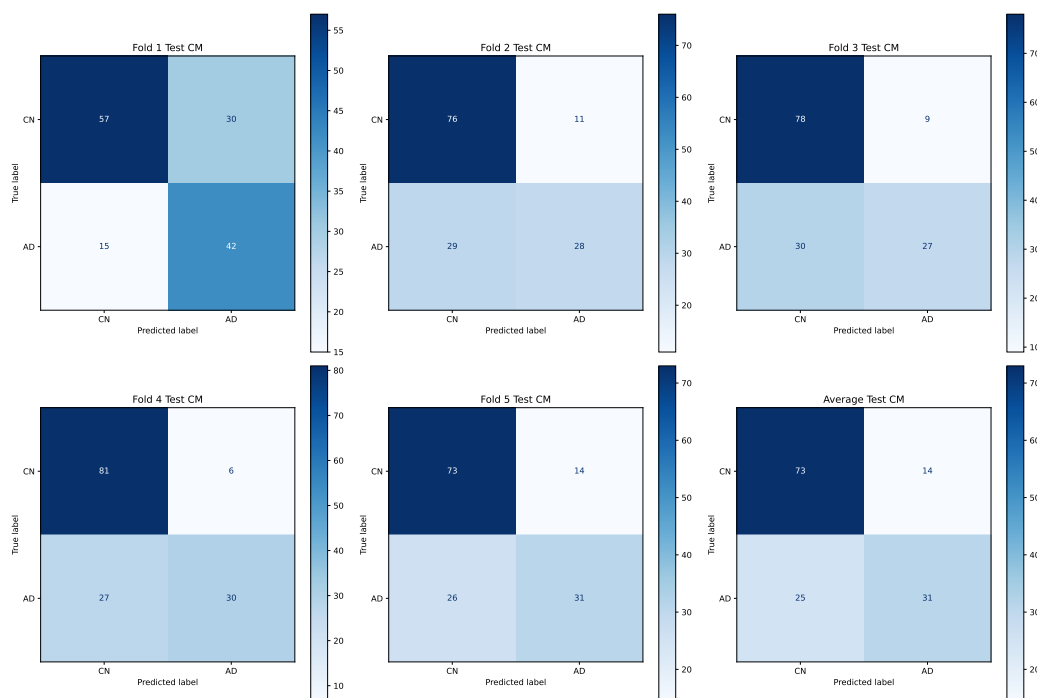
(b) Validation loss average and each of the 5 folds

Figure 31: ResNet-50 36 resolution Training Performance Metrics (Part 1)

Figure 32 (a) also shows the validation accuracy for each of the 5 folds and figure 32 (b) shows the confusion matrix for each of the 5 folds along with the average (test) metrics on the bottom right corner.



(a) Validation accuracy average and each of the 5 folds

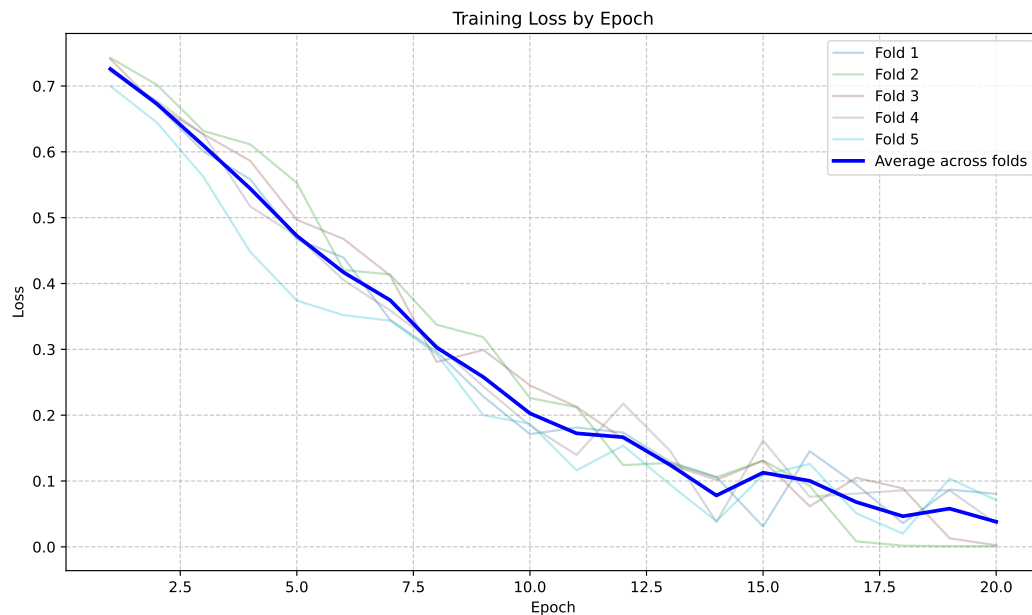


(b) Confusion Matrix for each of the 5 folds and the test set matrix

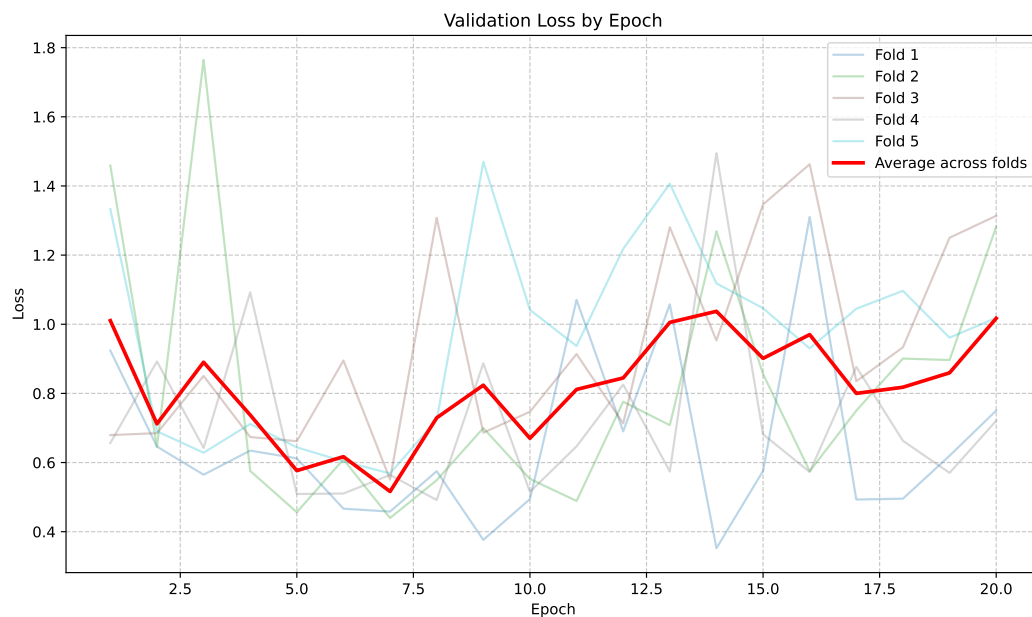
Figure 32: ResNet-50 36 Training Performance Metrics (Part 2)

A.3 ResNet-34 72x72x72 Graphs

Figure 33 below show the training and validation loss curves for each of the 5 folds for cross validation.



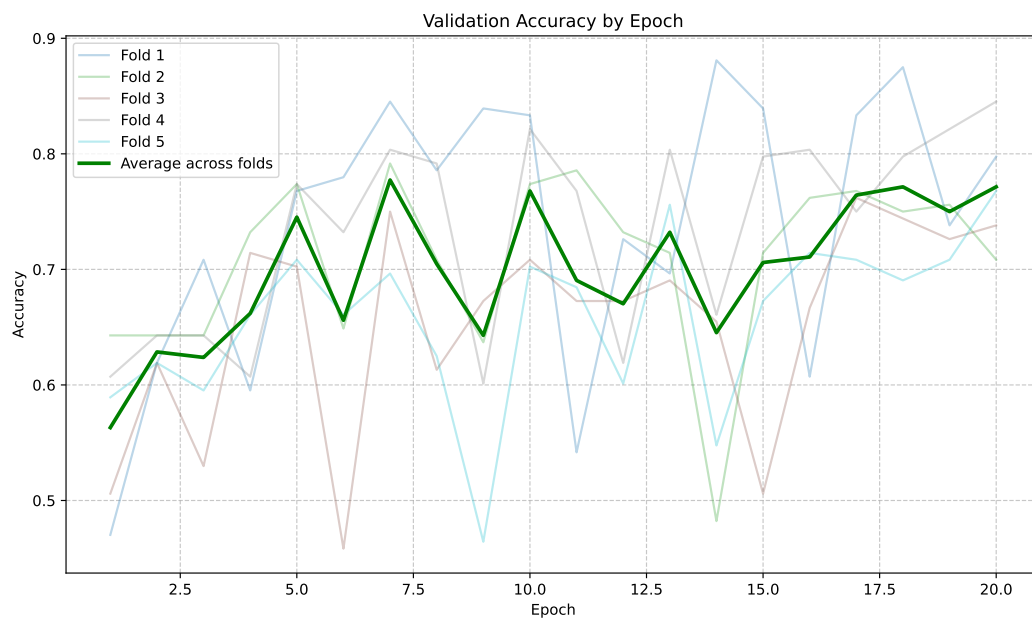
(a) Training loss average and each of the 5 folds



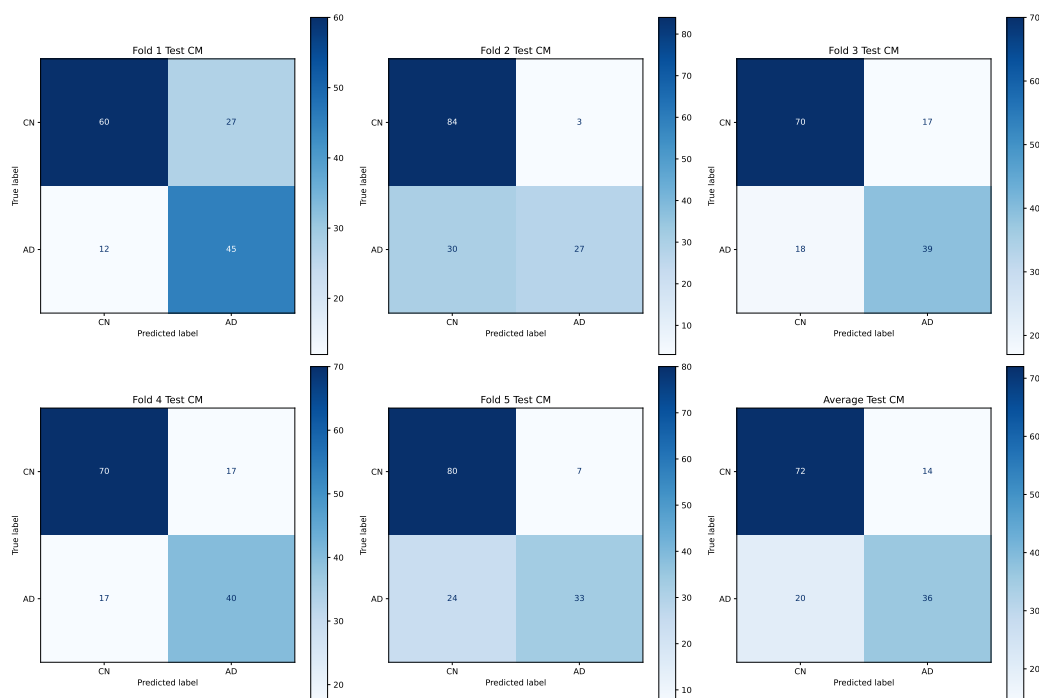
(b) Validation loss average and each of the 5 folds

Figure 33: ResNet-34 72 Training Performance Metrics (Part 1)

Figure 34 (a) also shows the validation accuracy for each of the 5 folds and figure 34 (b) shows the confusion matrix for each of the 5 folds along with the average (test) metrics on the bottom right corner.



(a) Validation accuracy average and each of the 5 folds

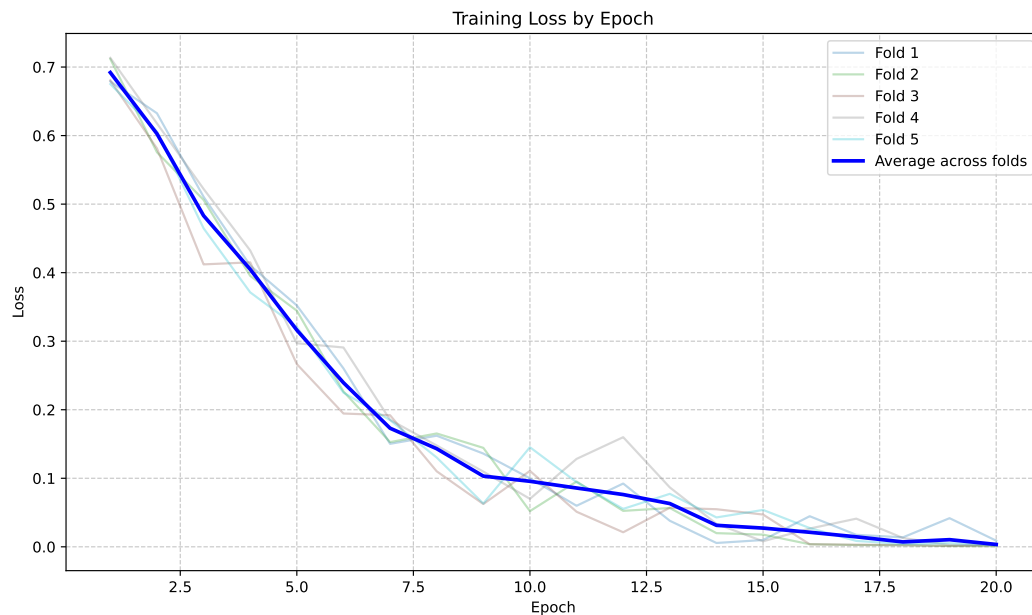


(b) Confusion Matrix for each of the 5 folds and the test set matrix

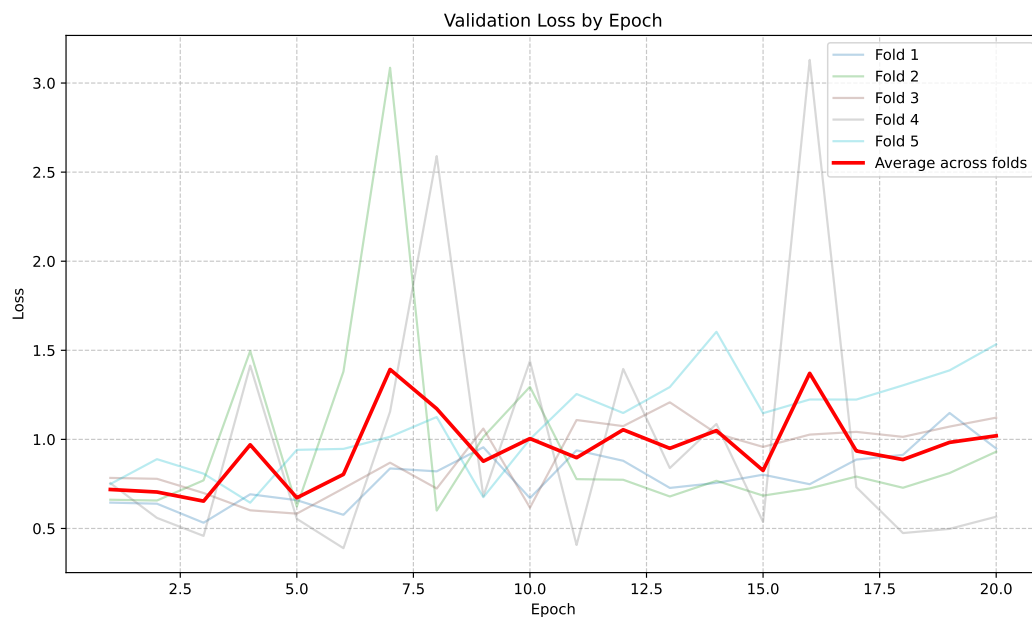
Figure 34: ResNet-34 Training Performance Metrics (Part 2)

A.4 DesnseNet-201 72x72x72 Graphs

Figure 35 below show the training and validation loss curves for each of the 5 folds for cross validation.



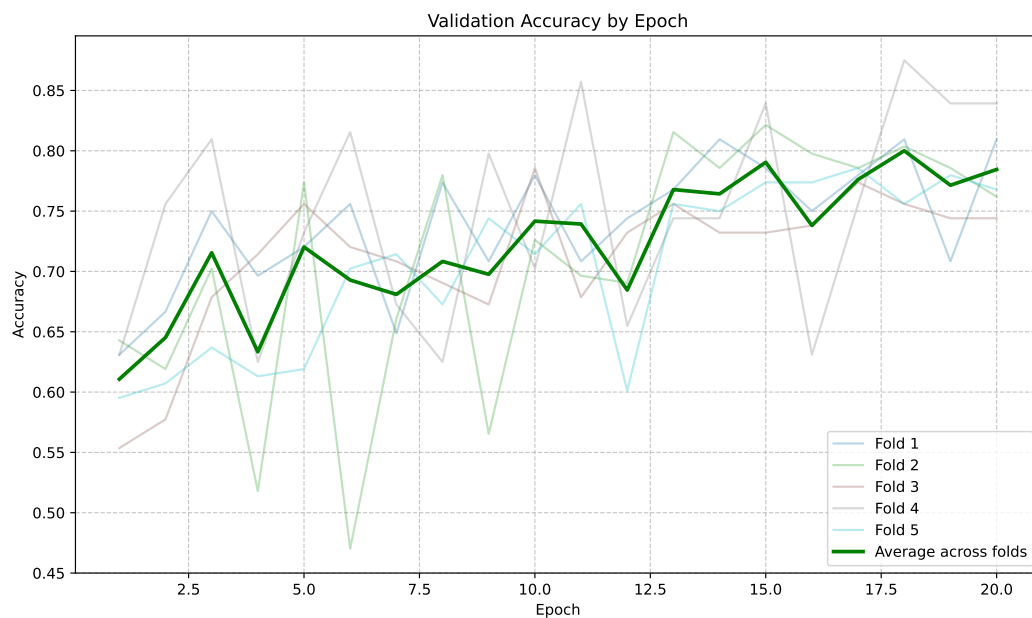
(a) Training loss average and each of the 5 folds



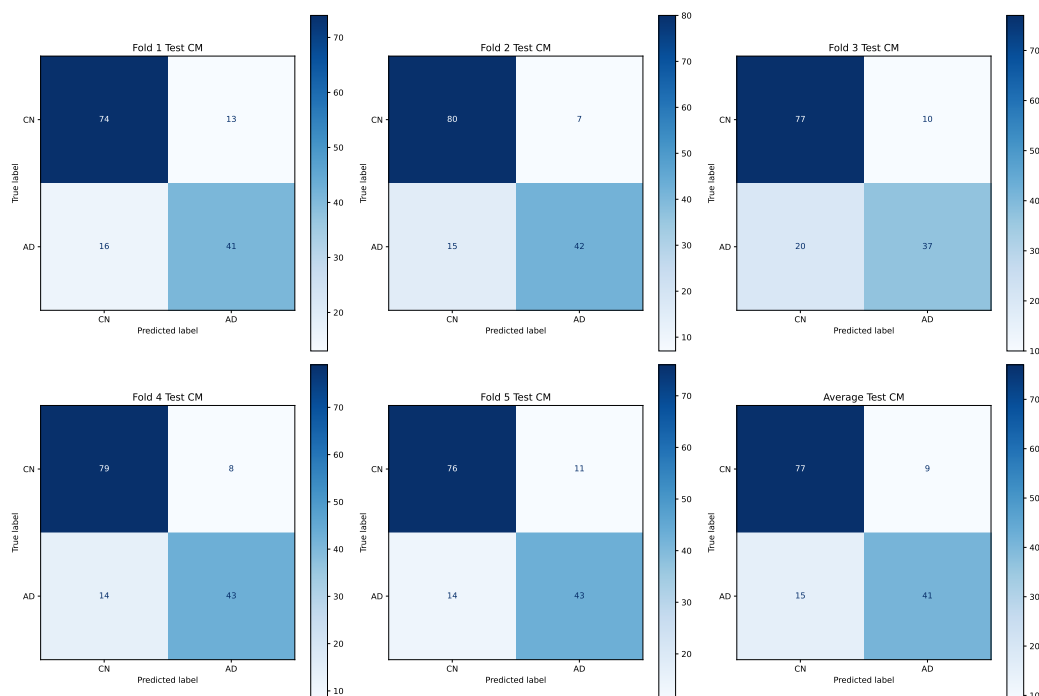
(b) Validation loss average and each of the 5 folds

Figure 35: DenseNet-201 Training Performance Metrics (Part 1)

Figure 36 (a) also shows the validation accuracy for each of the 5 folds and figure 36 (b) shows the confusion matrix for each of the 5 folds along with the average (test) metrics on the bottom right corner.



(a) Validation accuracy average and each of the 5 folds

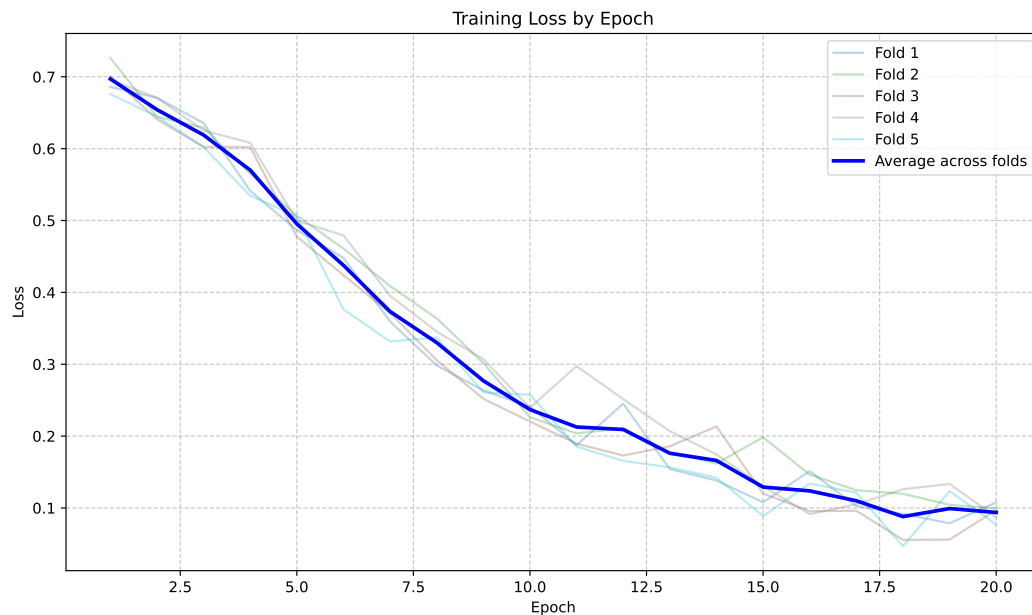


(b) Confusion Matrix for each of the 5 folds and the test set matrix

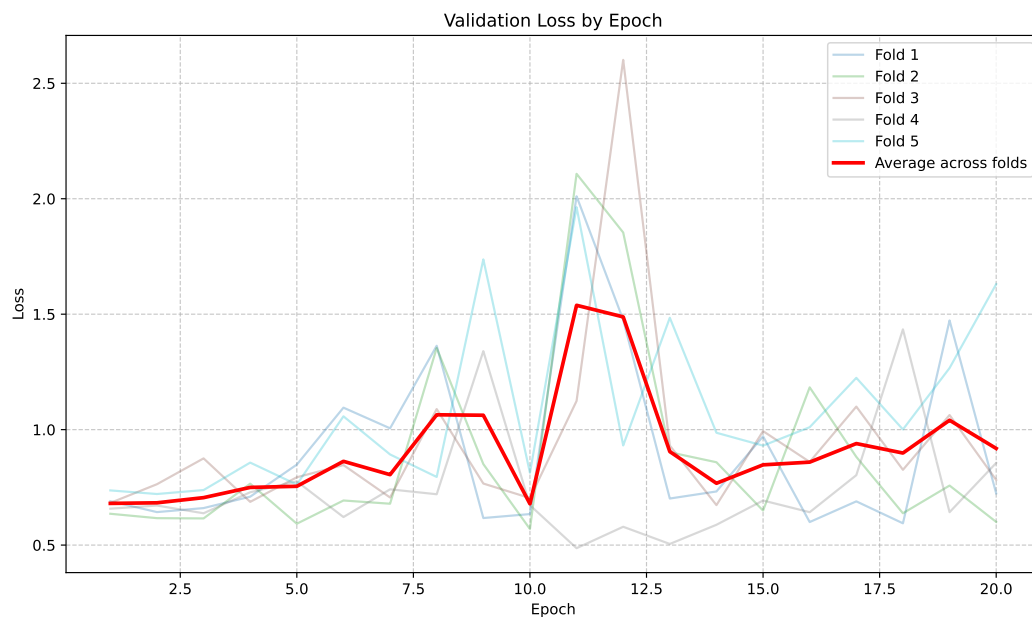
Figure 36: DenseNet-201 Training Performance Metrics (Part 2)

A.5 DesnseNet-201 36x36x36 Graphs

Figure 37 below show the training and validation loss curves for each of the 5 folds for cross validation.



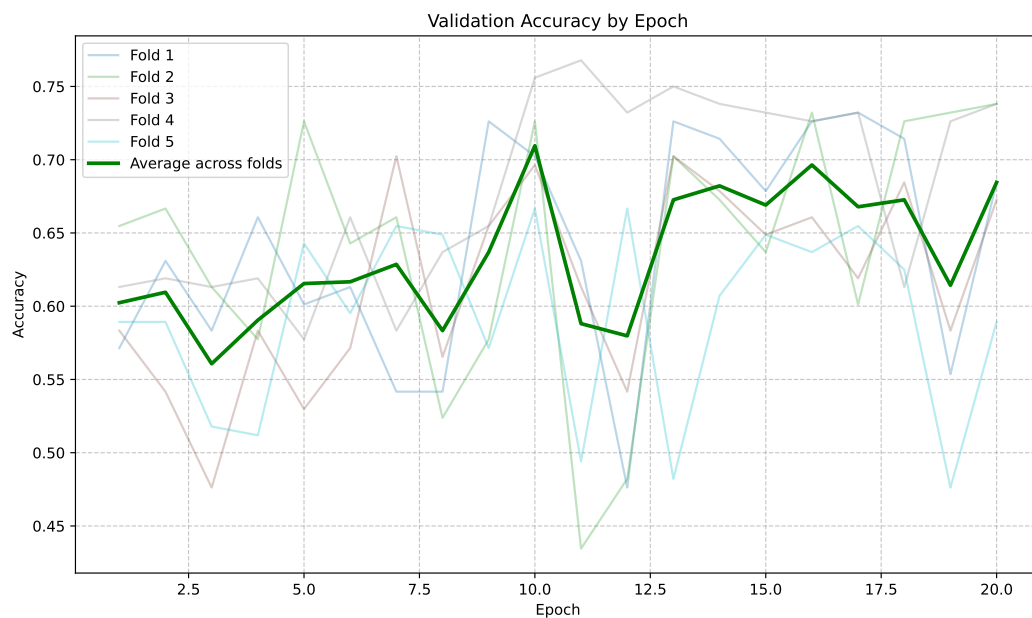
(a) Training loss average and each of the 5 folds



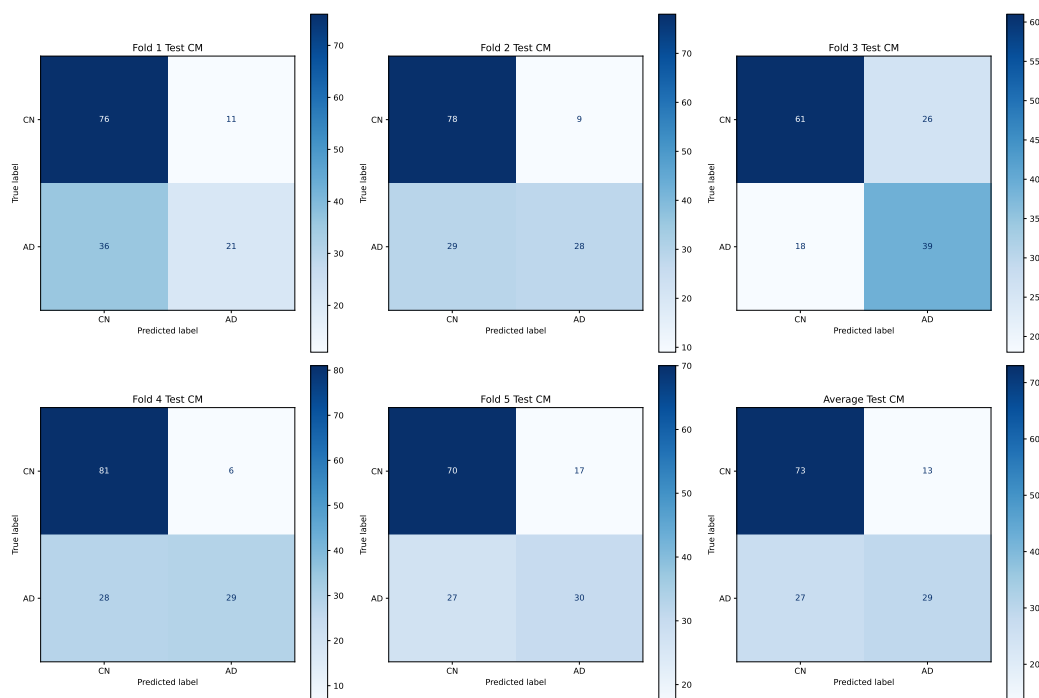
(b) Validation loss average and each of the 5 folds

Figure 37: DenseNet-201 Training Performance Metrics (Part 1)

Figure 38 (a) also shows the validation accuracy for each of the 5 folds and figure 38 (b) shows the confusion matrix for each of the 5 folds along with the average (test) metrics on the bottom right corner.



(a) Validation accuracy average and each of the 5 folds

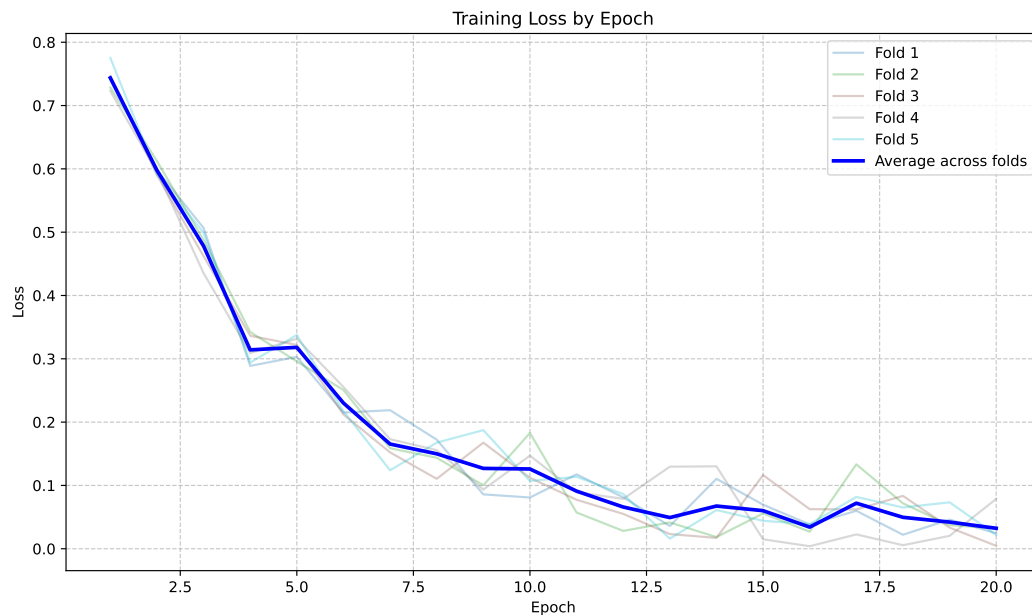


(b) Confusion Matrix for each of the 5 folds and the test set matrix

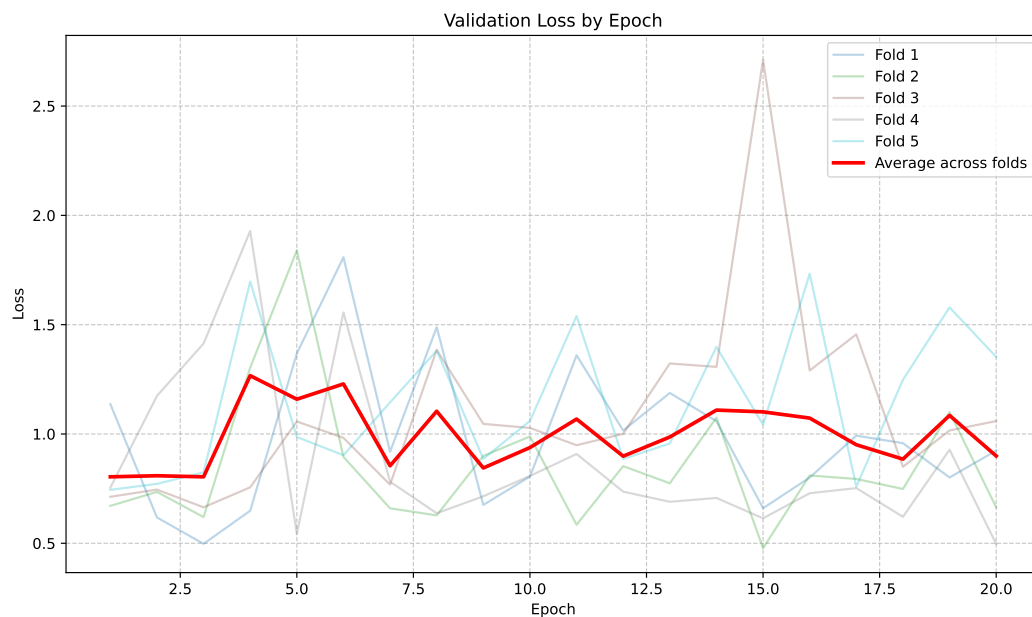
Figure 38: DenseNet-201 Training Performance Metrics (Part 2)

A.6 SENet 72x72x72 Graphs

Figure 39 below show the training and validation loss curves for each of the 5 folds for cross validation.



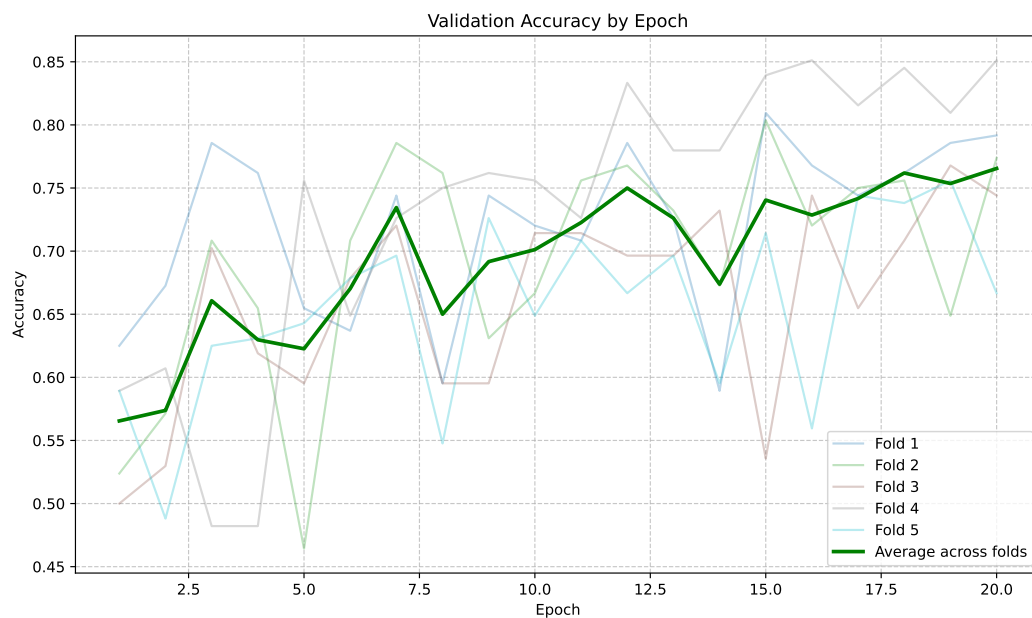
(a) Training loss average and each of the 5 folds



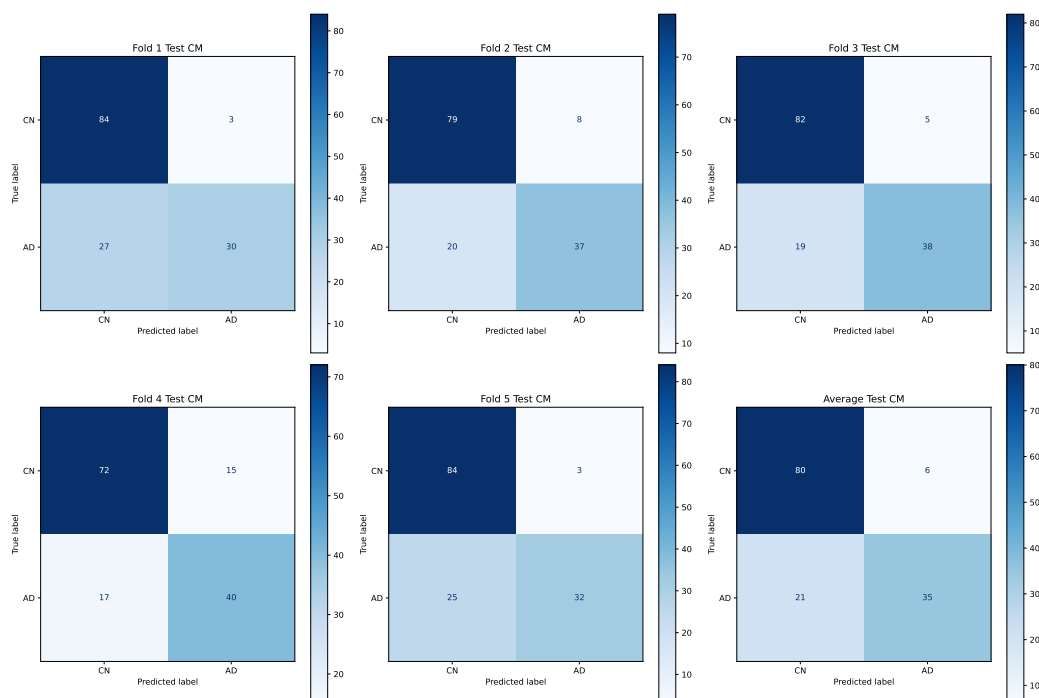
(b) Validation loss average and each of the 5 folds

Figure 39: SENet (72) Training Performance Metrics (Part 1)

Figure 40 (a) also shows the validation accuracy for each of the 5 folds and figure 40 (b) shows the confusion matrix for each of the 5 folds along with the average (test) metrics on the bottom right corner.



(a) Validation accuracy average and each of the 5 folds

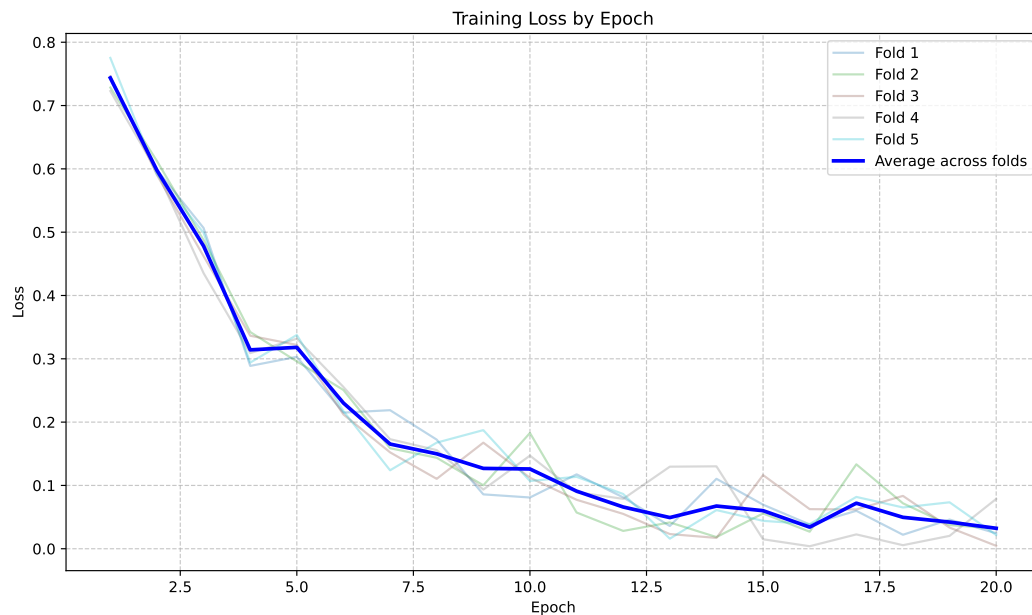


(b) Confusion Matrix for each of the 5 folds and the test set matrix

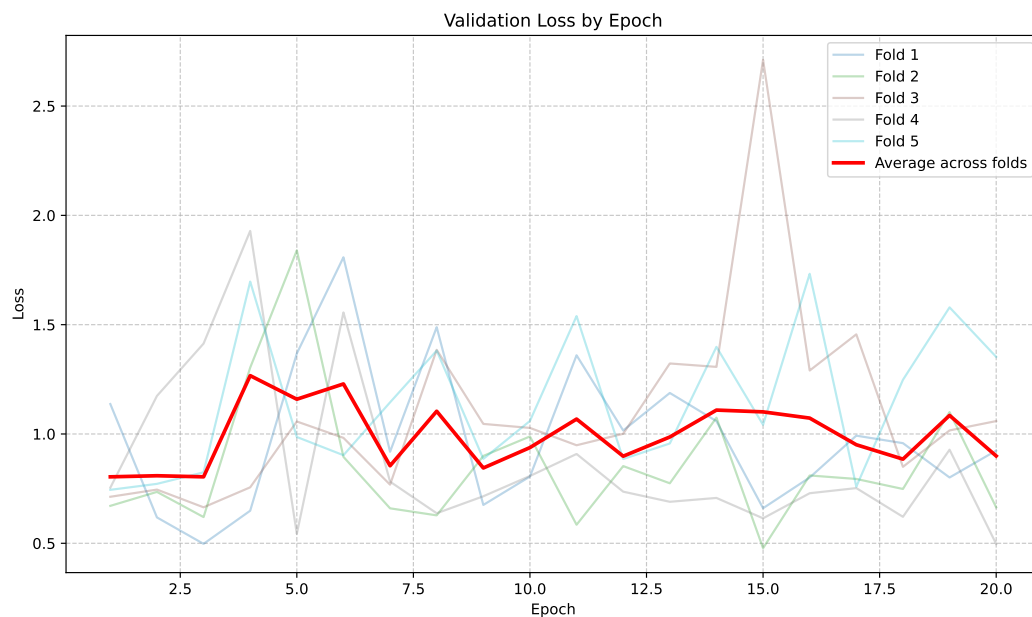
Figure 40: SENet (72) Training Performance Metrics (Part 2)

A.7 SENet 36x36x36 Graphs

Figure 41 below show the training and validation loss curves for each of the 5 folds for cross validation.



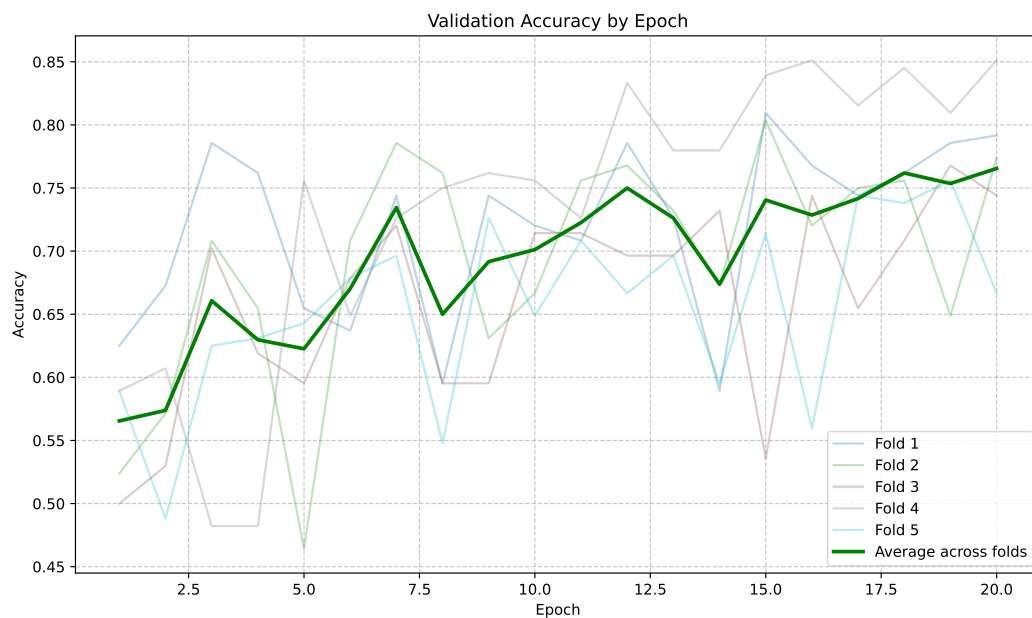
(a) Training loss average and each of the 5 folds



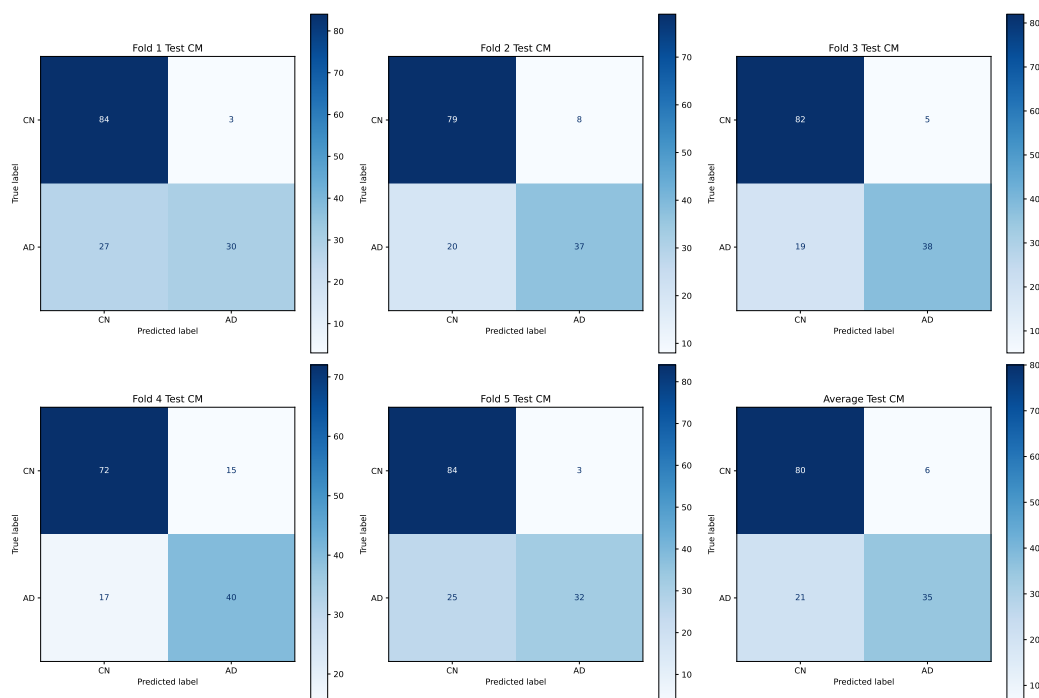
(b) Validation loss average and each of the 5 folds

Figure 41: SENet (36) Training Performance Metrics (Part 1)

Figure 42 (a) also shows the validation accuracy for each of the 5 folds and figure 42 (b) shows the confusion matrix for each of the 5 folds along with the average (test) metrics on the bottom right corner.



(a) Validation accuracy average and each of the 5 folds



(b) Confusion Matrix for each of the 5 folds and the test set matrix

Figure 42: SENet (36) Training Performance Metrics (Part 2)